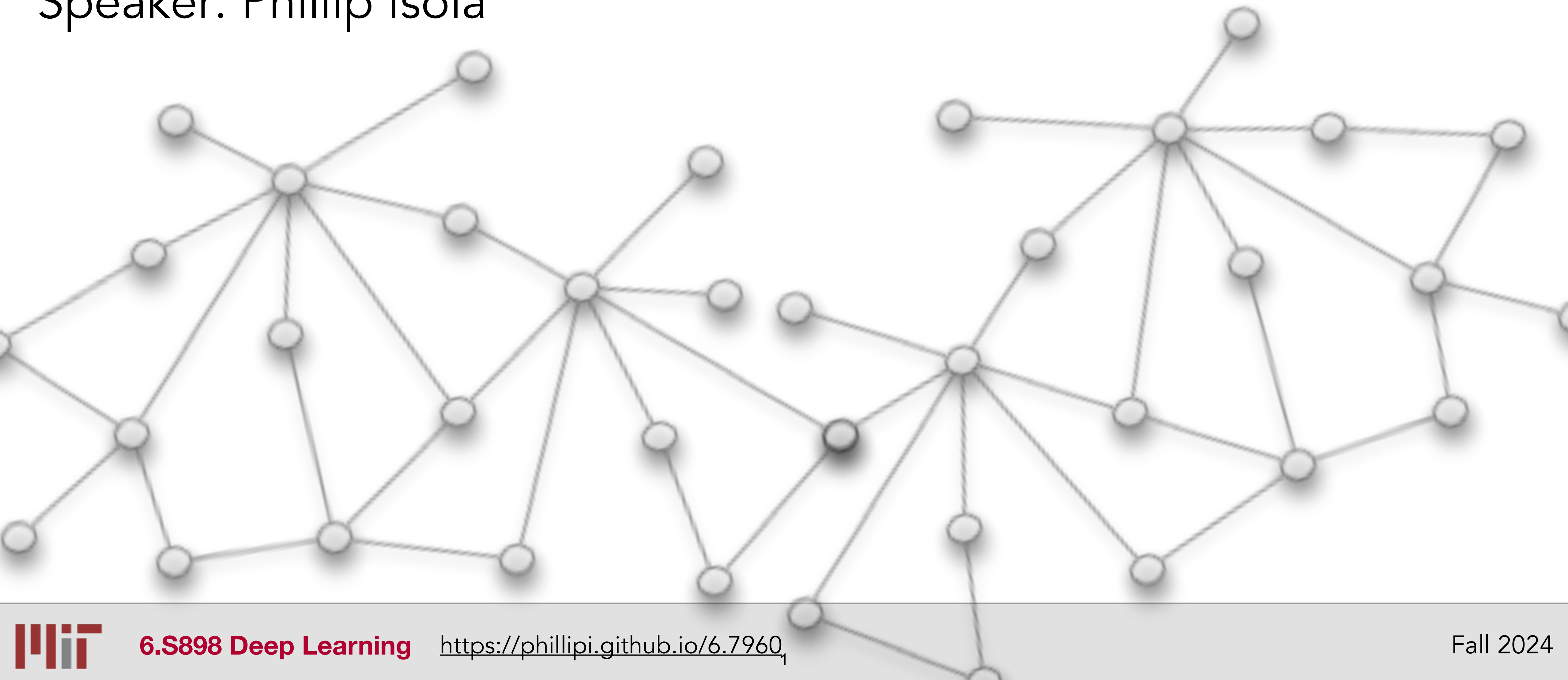


Lecture 5: Graph Neural Networks

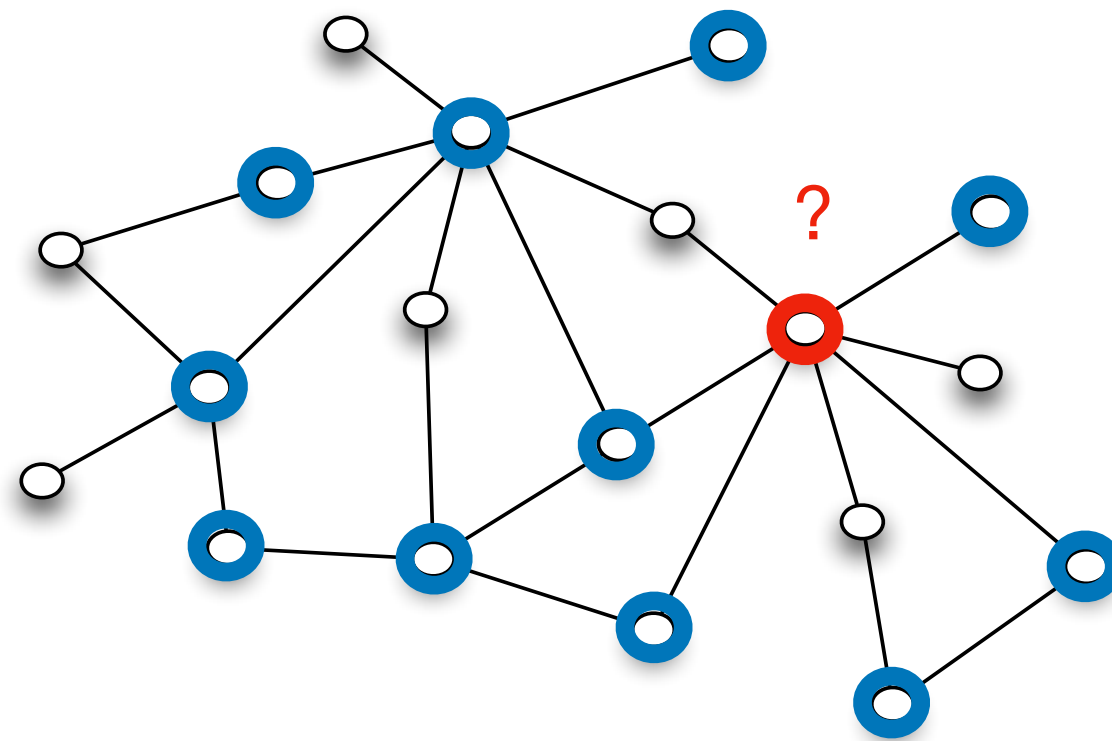
Speaker: Phillip Isola



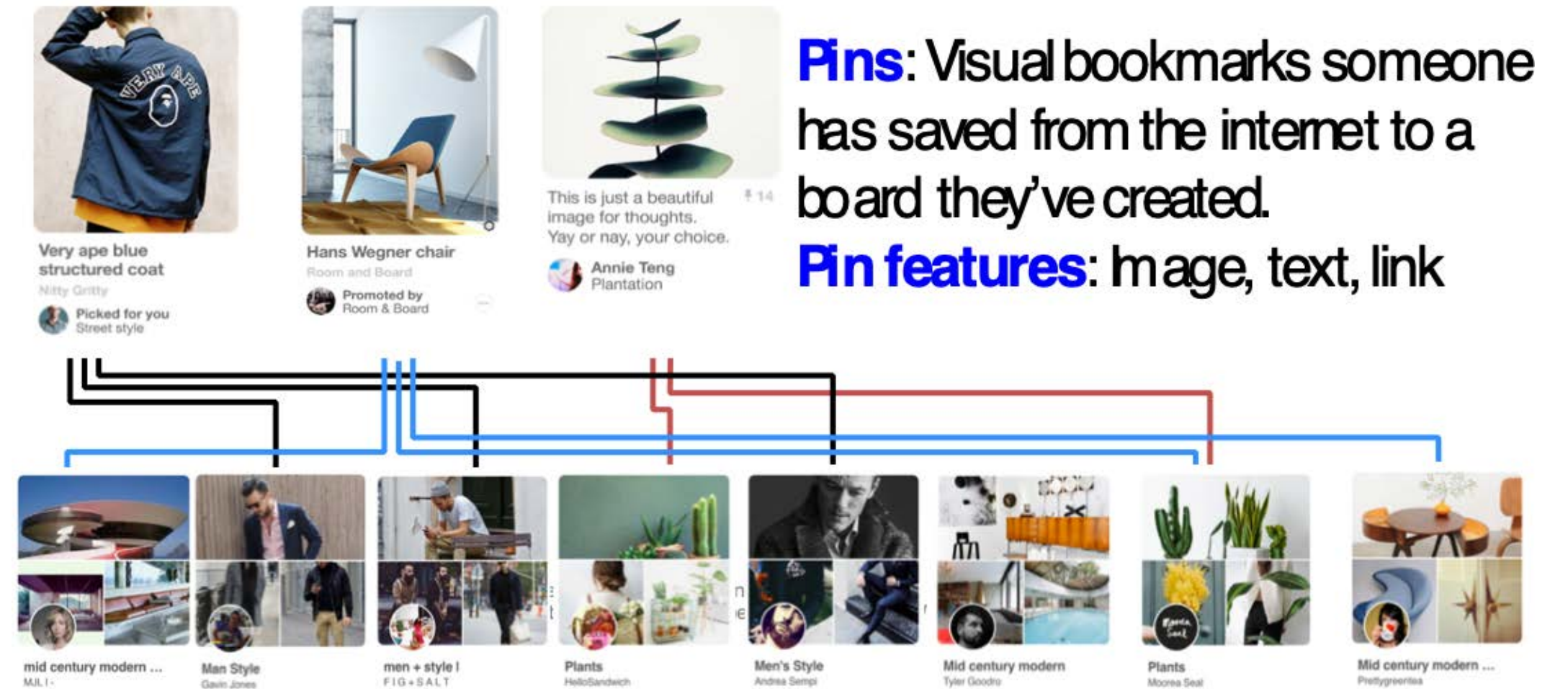
Roadmap

- Learning tasks with graphs
- Message passing GNNs
- Approximation Power

Prediction with graphs: examples



**Node
classification**



Boards

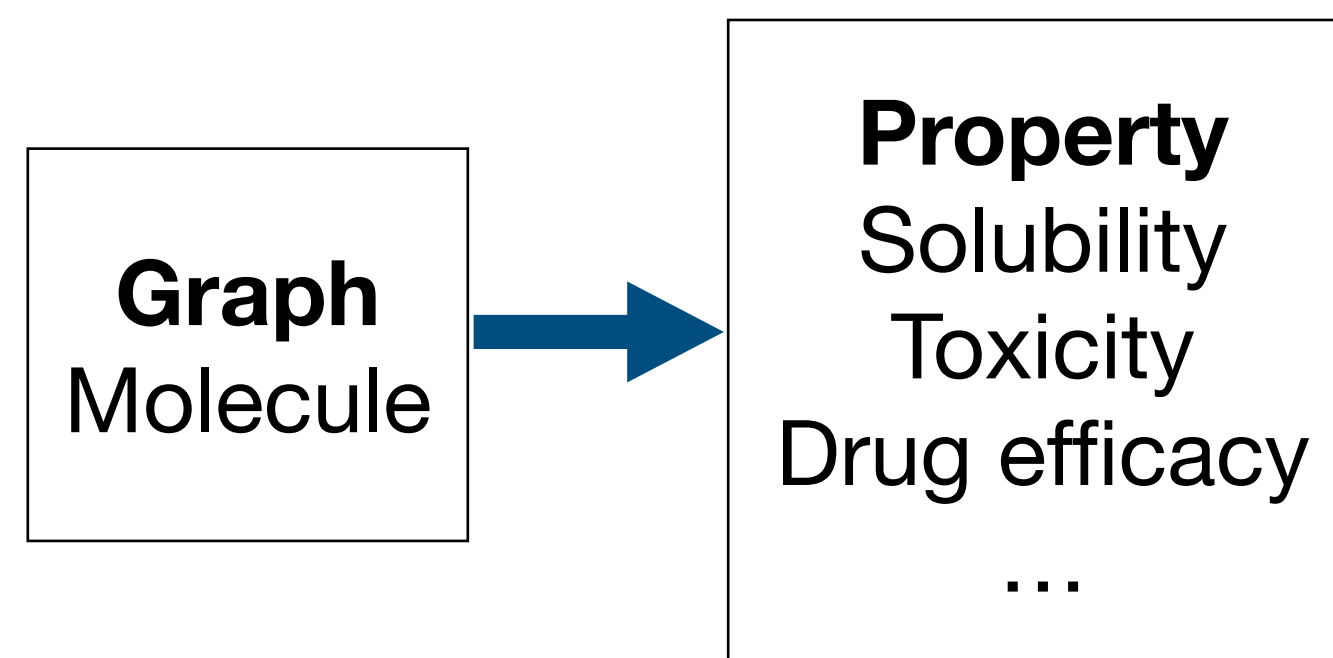
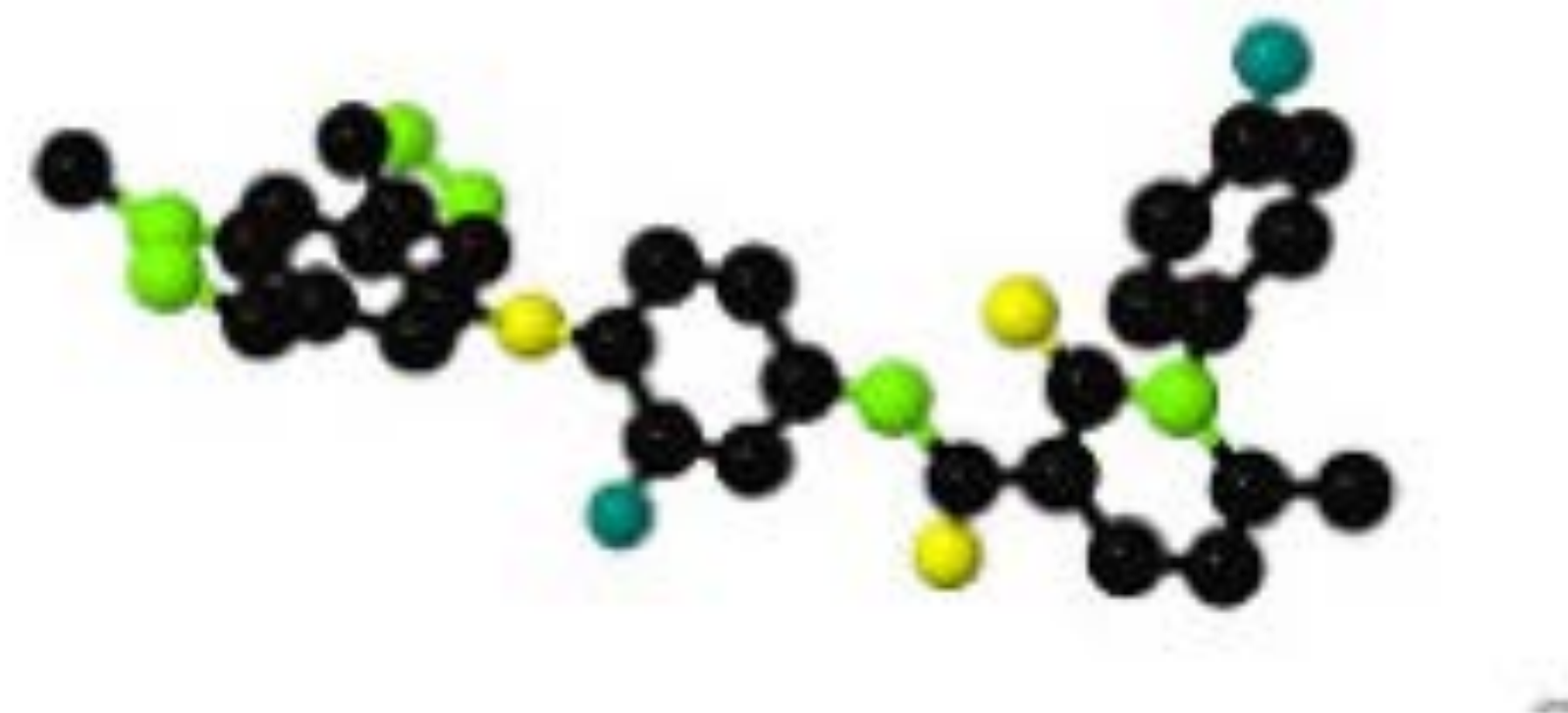
Link Prediction

(e.g. Ying et al, 2018; illustration: J. Leskovec)

Illustration © J. Leskovec. Text © Derrow-Pinion, et al. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>

concerned recommender systems, which are very naturally representable as a graph-structured task: with Pinterest being one of the most early adopters [18, 33]. GNNs have also been deployed for product recommendation at Amazon [12], E-commerce applications at Alibaba [32], engagement forecasting and friend ranking in Snapchat [22, 24], and most relevantly, they are powering traffic predictions within Baidu Maps [6].

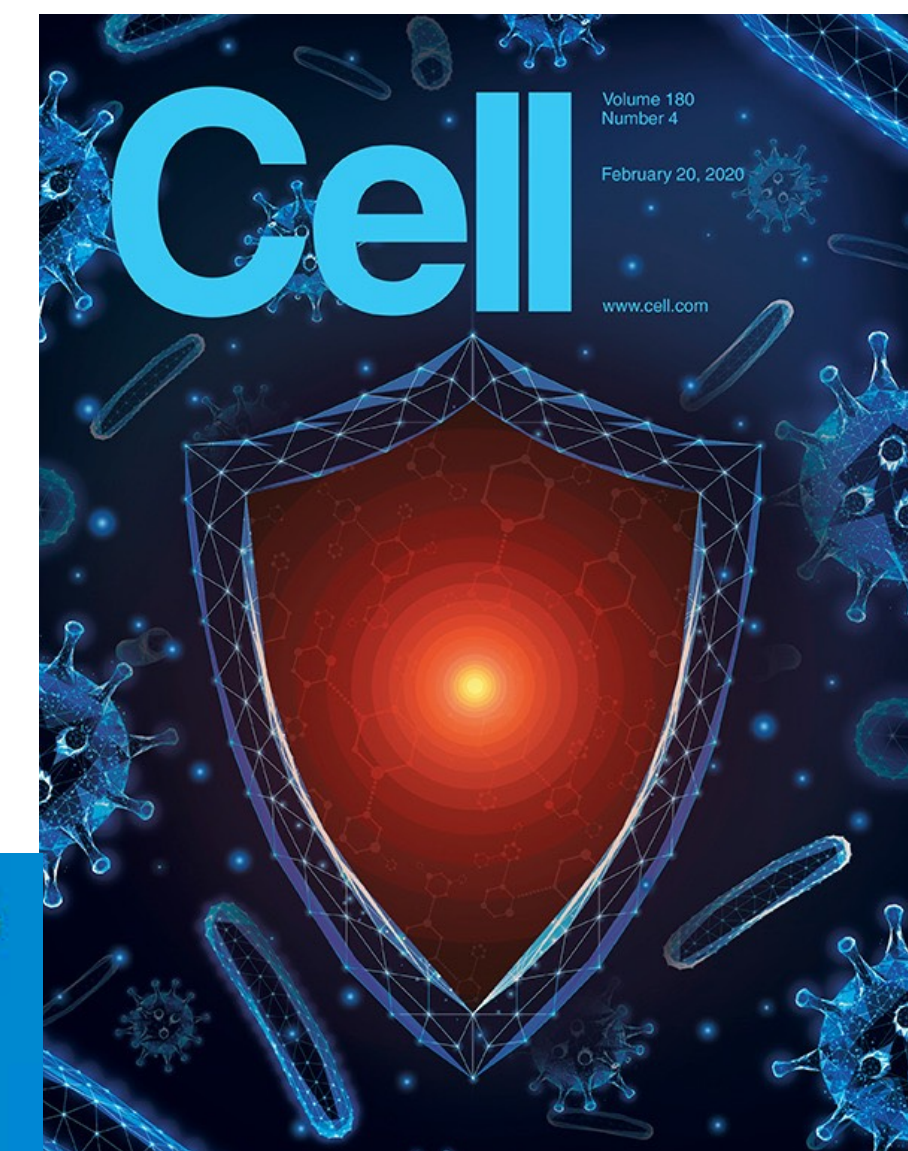
Example: molecule property prediction



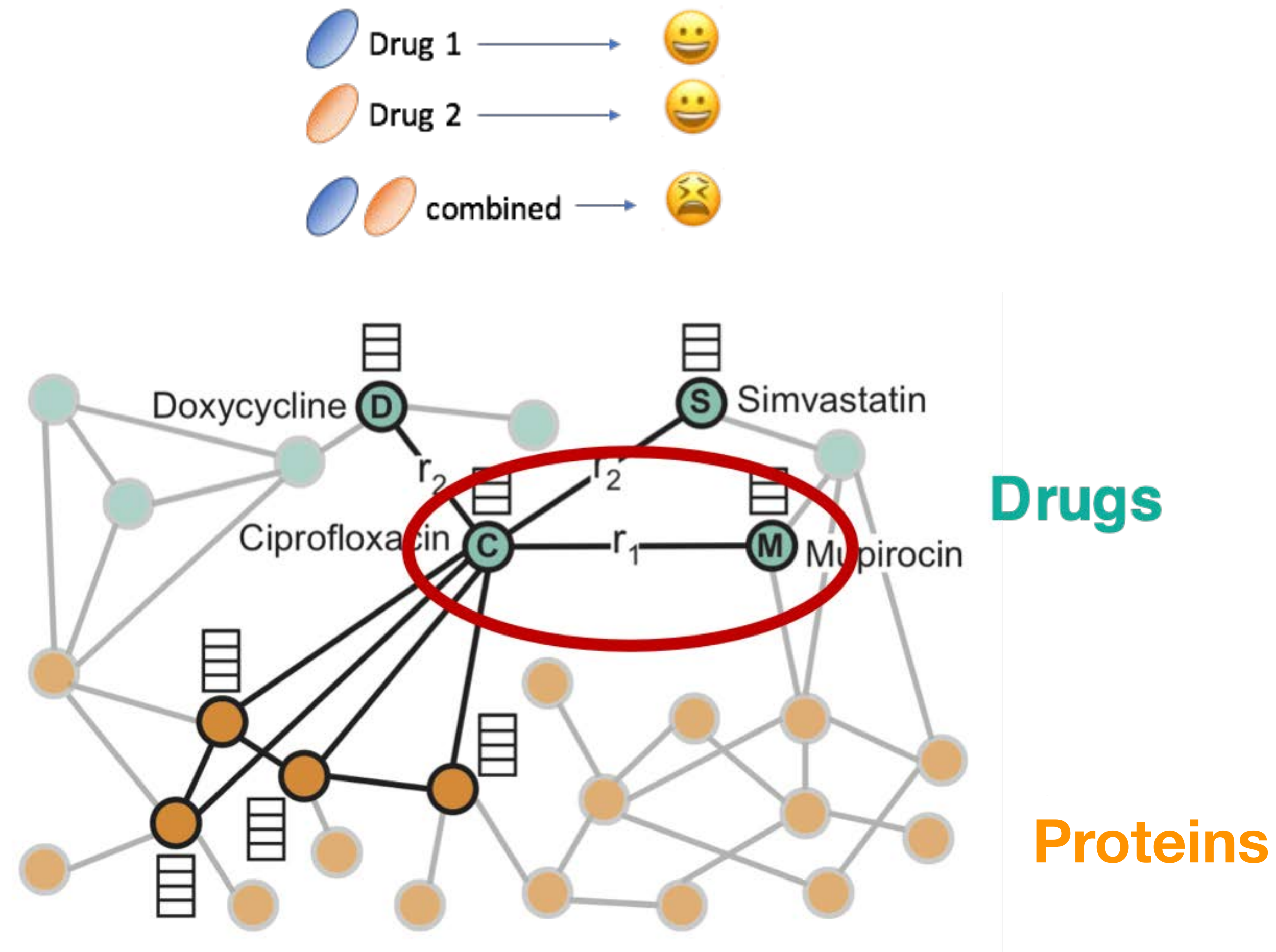
(Duvenaud et al, 2015, Stokes et al 2020,...)

Above © Enzymlogica on Flickr. Right © Cell Press. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>

On the cover: Antibiotic resistance is a pervasive public health problem, requiring the adoption of creative approaches to drug discovery. In this issue, ... [Show more](#)



Example: Polypharmacy side effects



Pair of Nodes
Drugs



Edge
Interaction type

© Zitnik, et al. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>

(Zitnik et al, 2018)

Example: Predicting traffic times

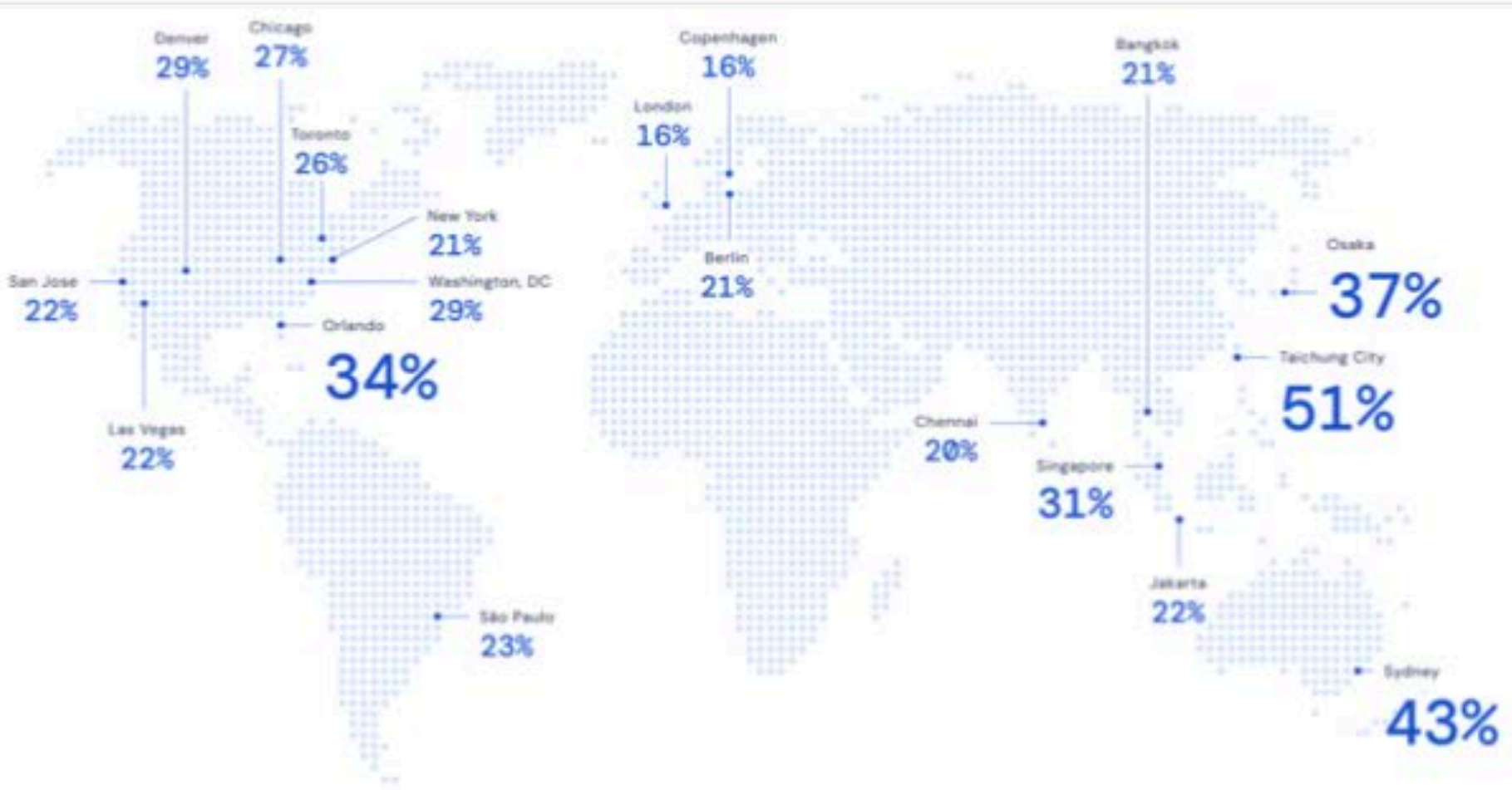
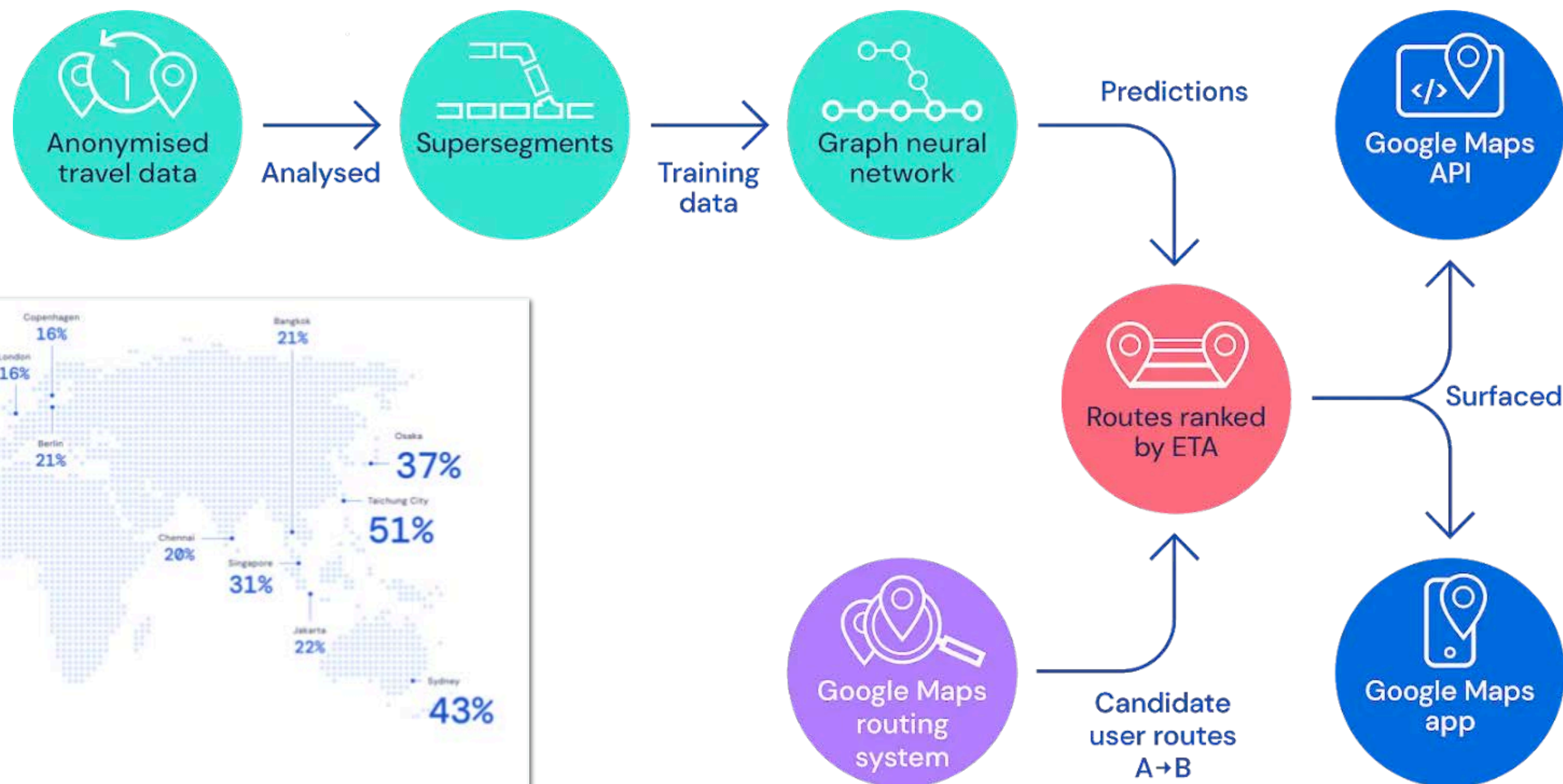
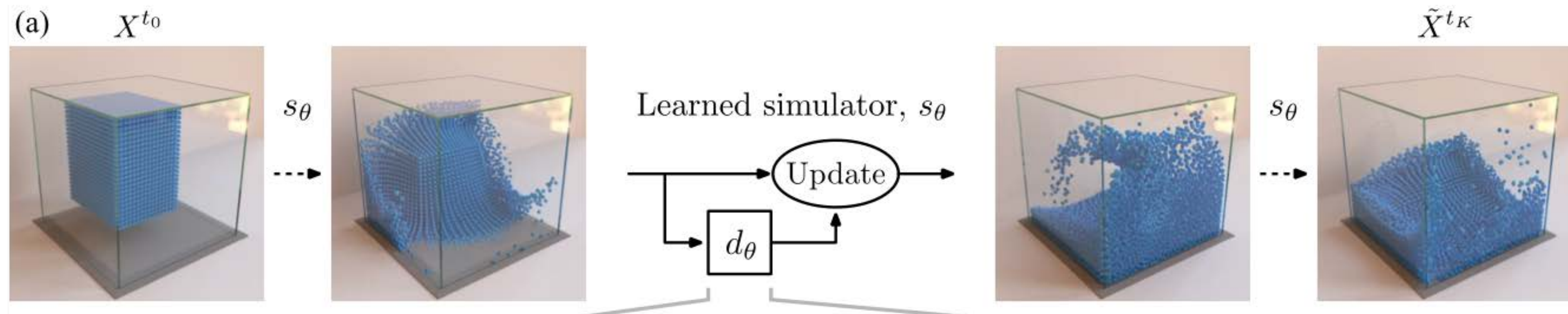


Figure 1: Google Maps estimated time-of-arrival (ETA) prediction improvements for several world regions, when using our deployed graph neural network-based estimator. Numbers represent relative reduction in negative ETA outcomes compared to the prior approach used in production. A negative ETA outcome occurs when the ETA error from the observed travel duration is over some threshold and acts as a

Figures © Paulo Estriga & Adam Cain. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>
<https://deepmind.com/blog/article/traffic-prediction-with-advanced-graph-neural-networks>

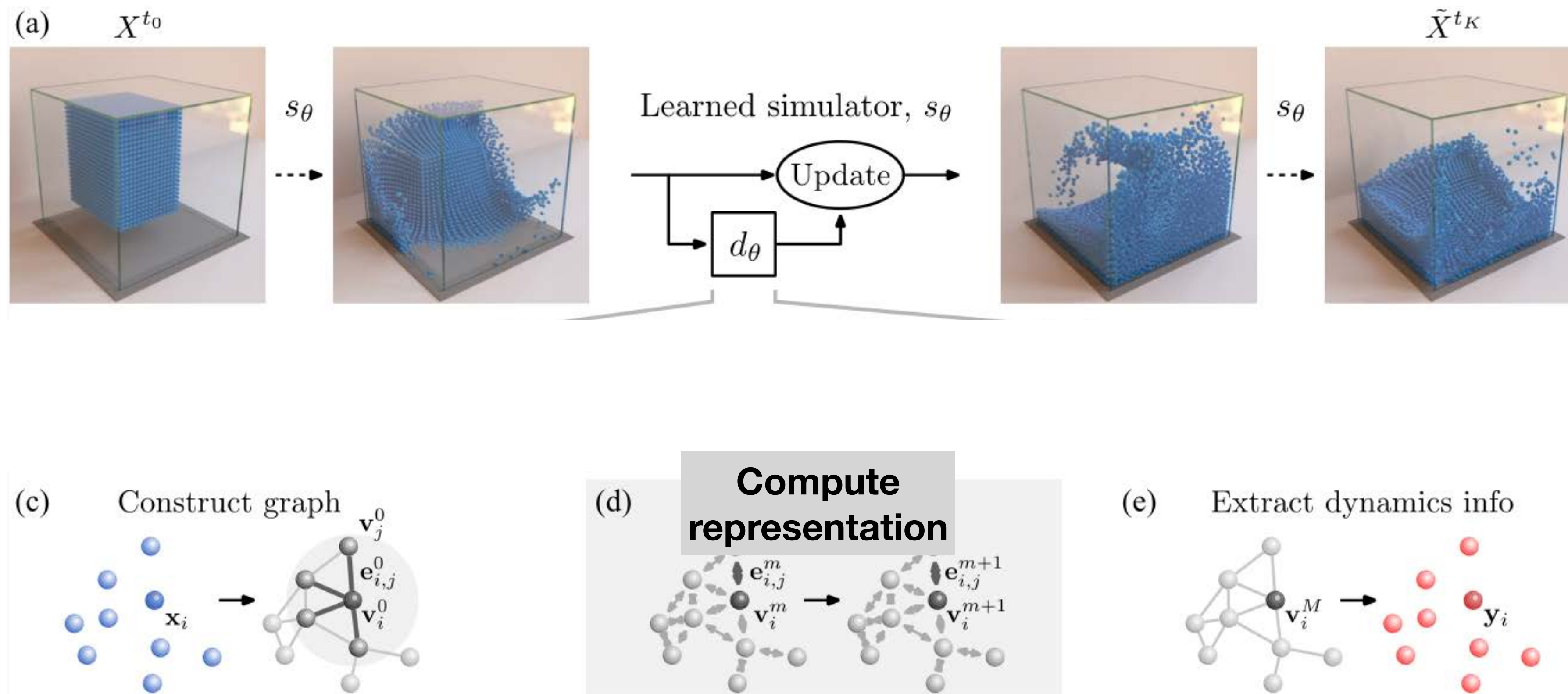
Example: learning to simulate physics



© Sanchez-Gonzalez, et al. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>

(Sanchez-Gonzalez et al, 2020)

Example: learning to simulate physics

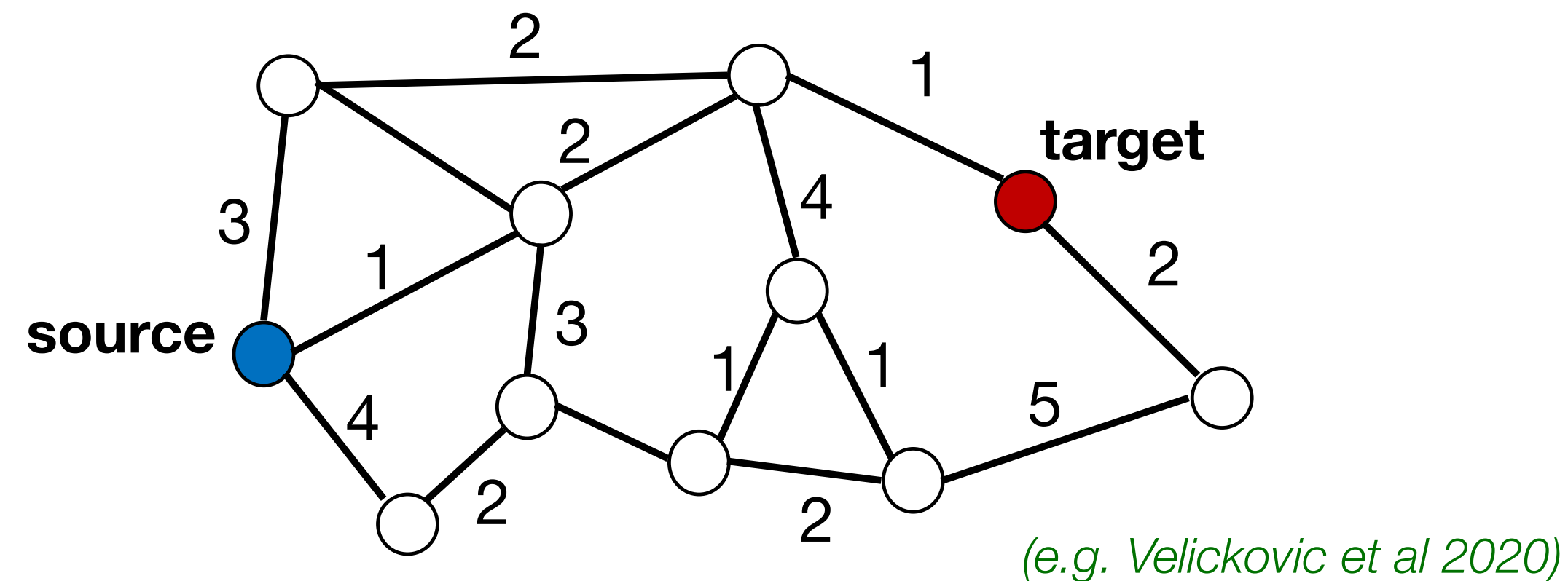


© Sanchez-Gonzalez, et al. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>

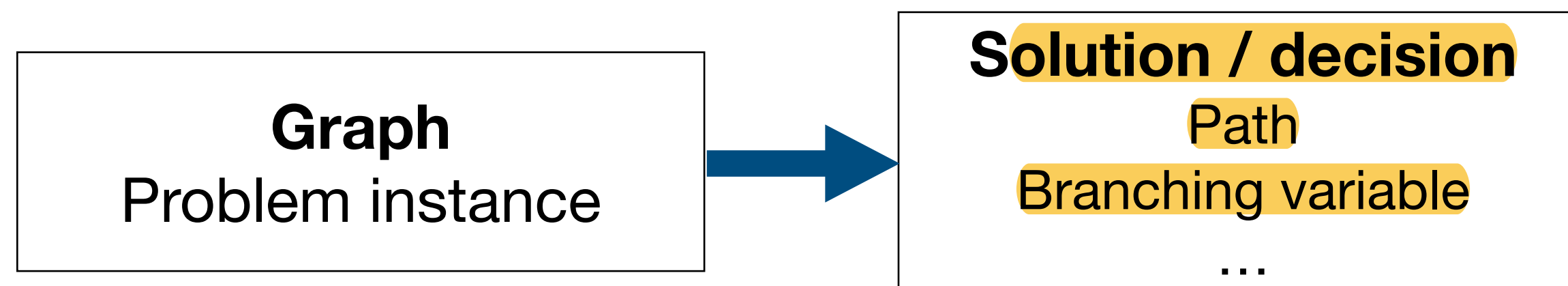
(Sanchez-Gonzalez et al, 2020)

Example: (Combinatorial) Optimization

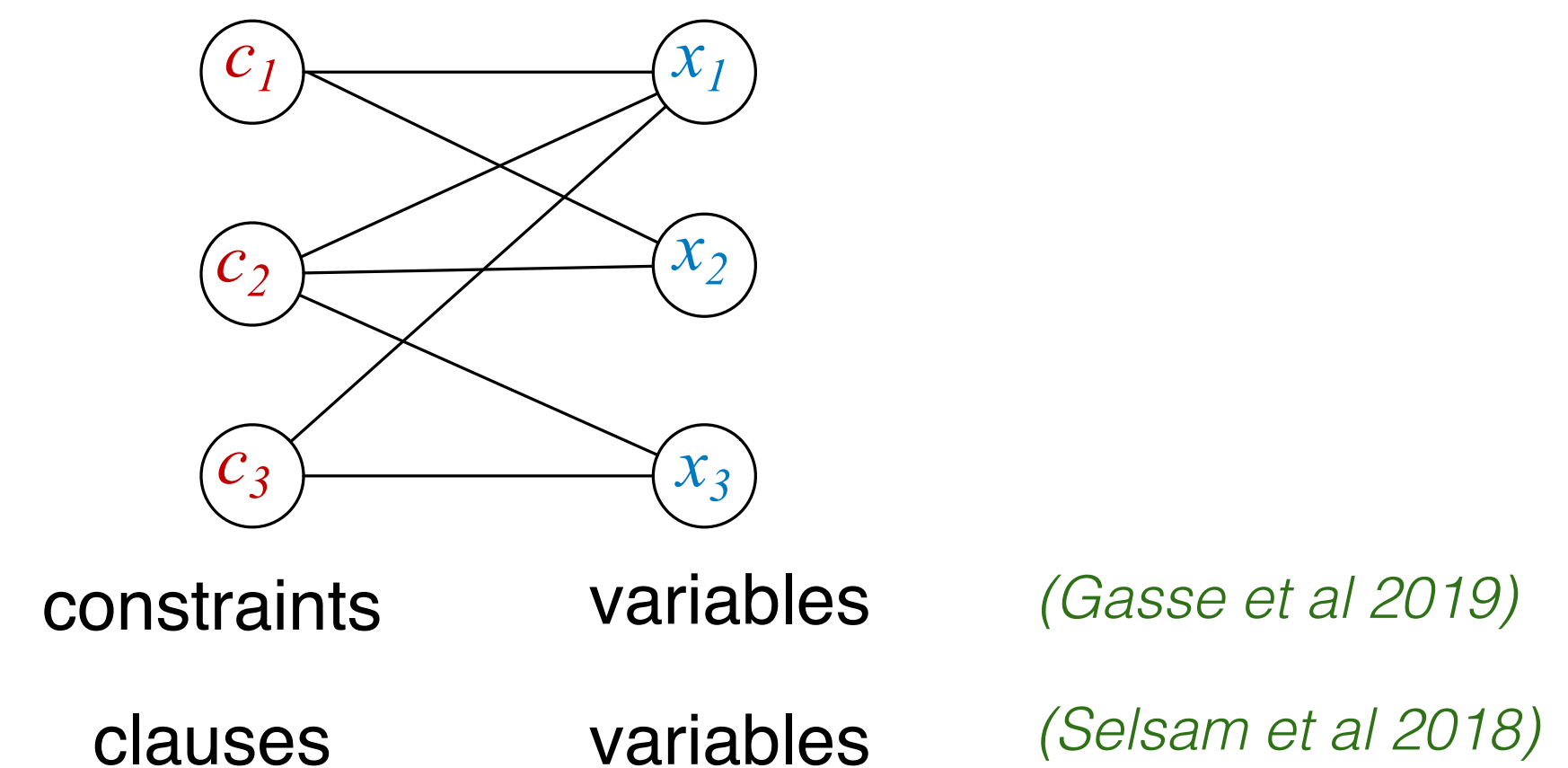
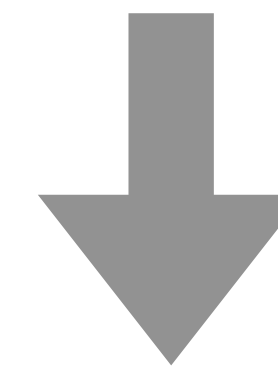
- replace full algorithm or learn steps (e.g. branching decision)



“Neural Algorithmic Reasoning”



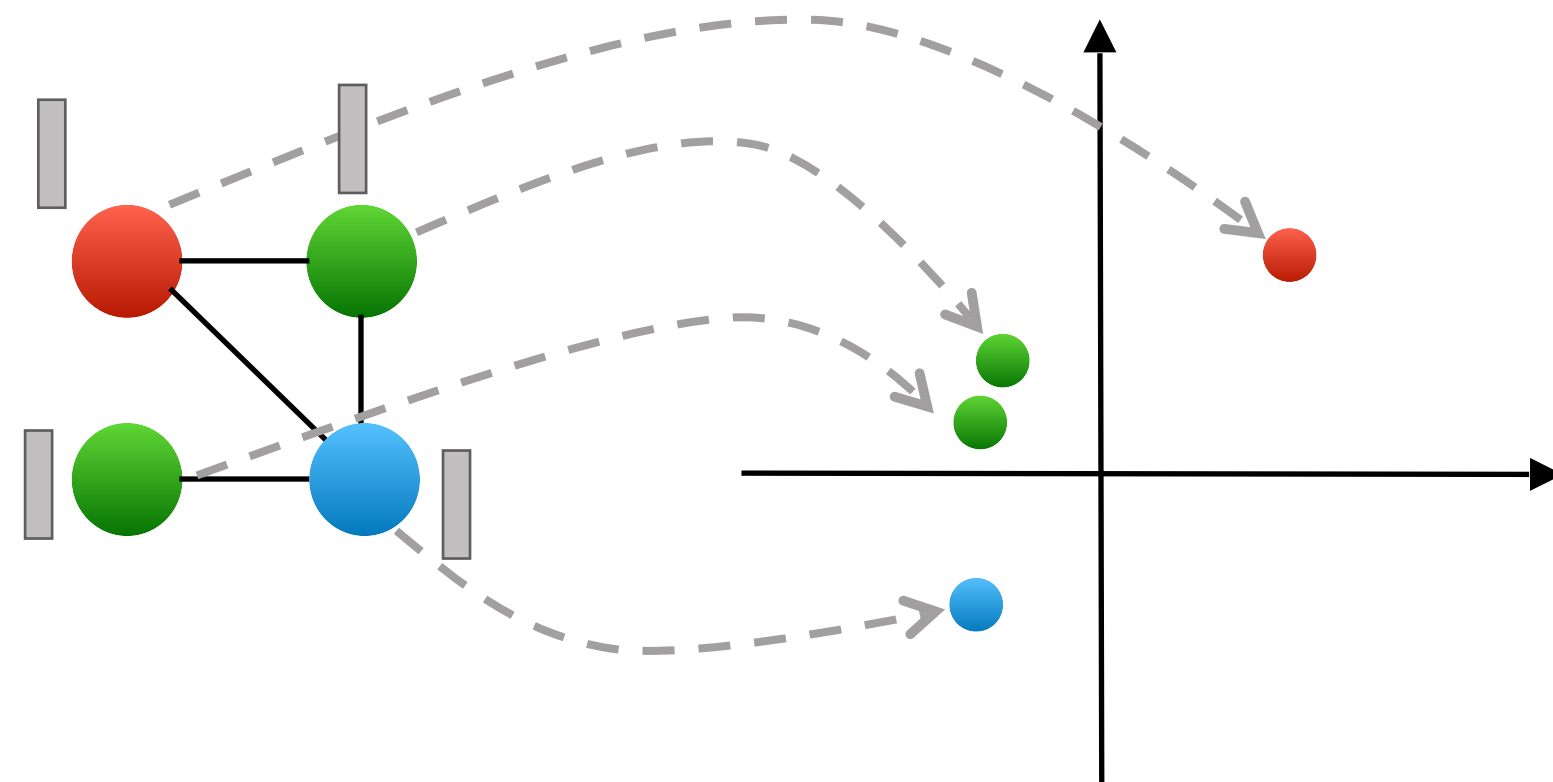
$$\begin{aligned} \min_x \quad & c^\top x \\ & Ax \leq b \\ & l \leq x \leq u \\ & x \in \mathbb{Z}^p \times \mathbb{R}^{n-p} \end{aligned}$$



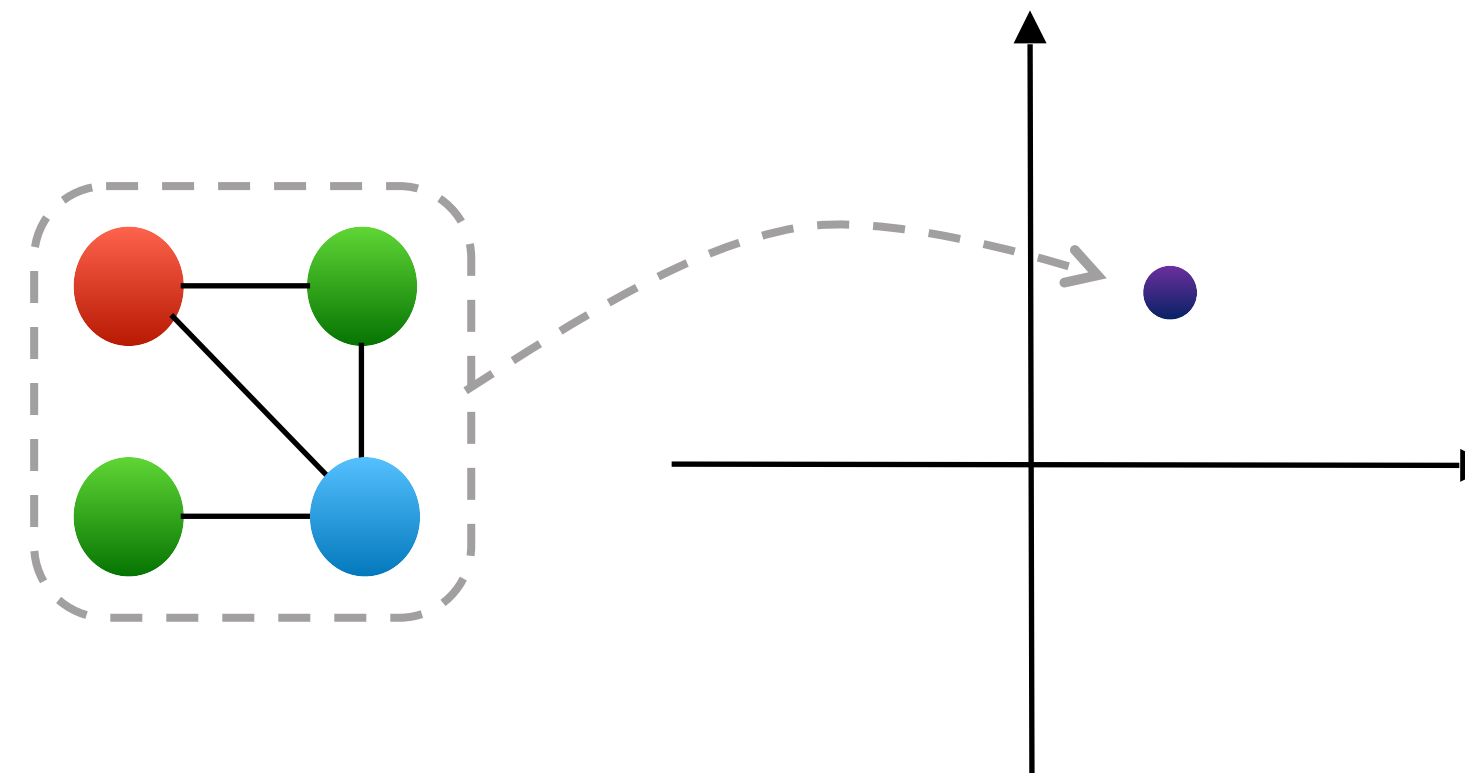
Two goals

Input: Graph + attribute vector for each node (adjacency matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$)

1. Node embeddings

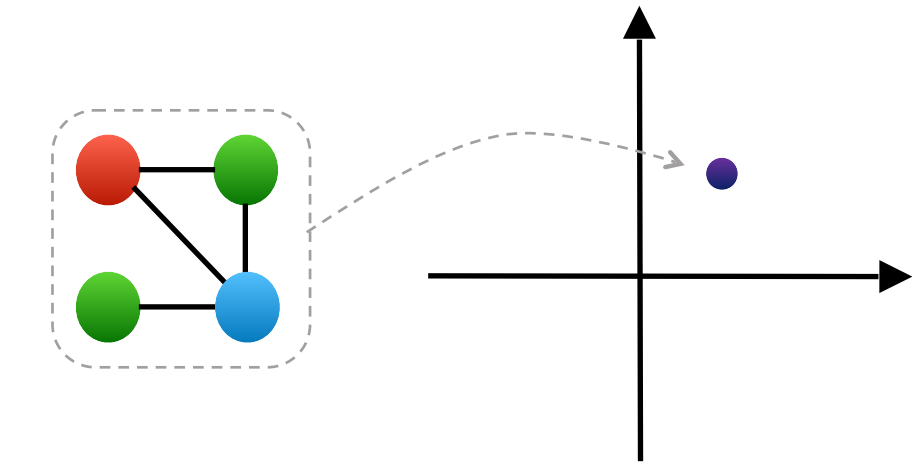


2. Graph embedding



GNNs: learn a **function** from graph/neighborhood + node/edge attributes to vector

Idea 1: fully-connected NN?



Idea 1: Use the adjacency matrix as input to a neural network

$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

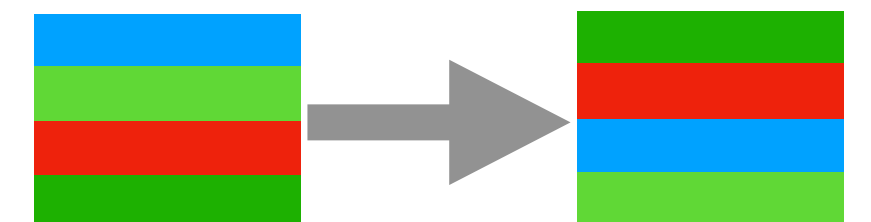
$$\mathbf{PAP}^\top = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

We want:

- **Permutation invariance (graph embedding):**
(output: single vector) and
- **Permutation equivariance (node embeddings):**
(output: one vector for each node)

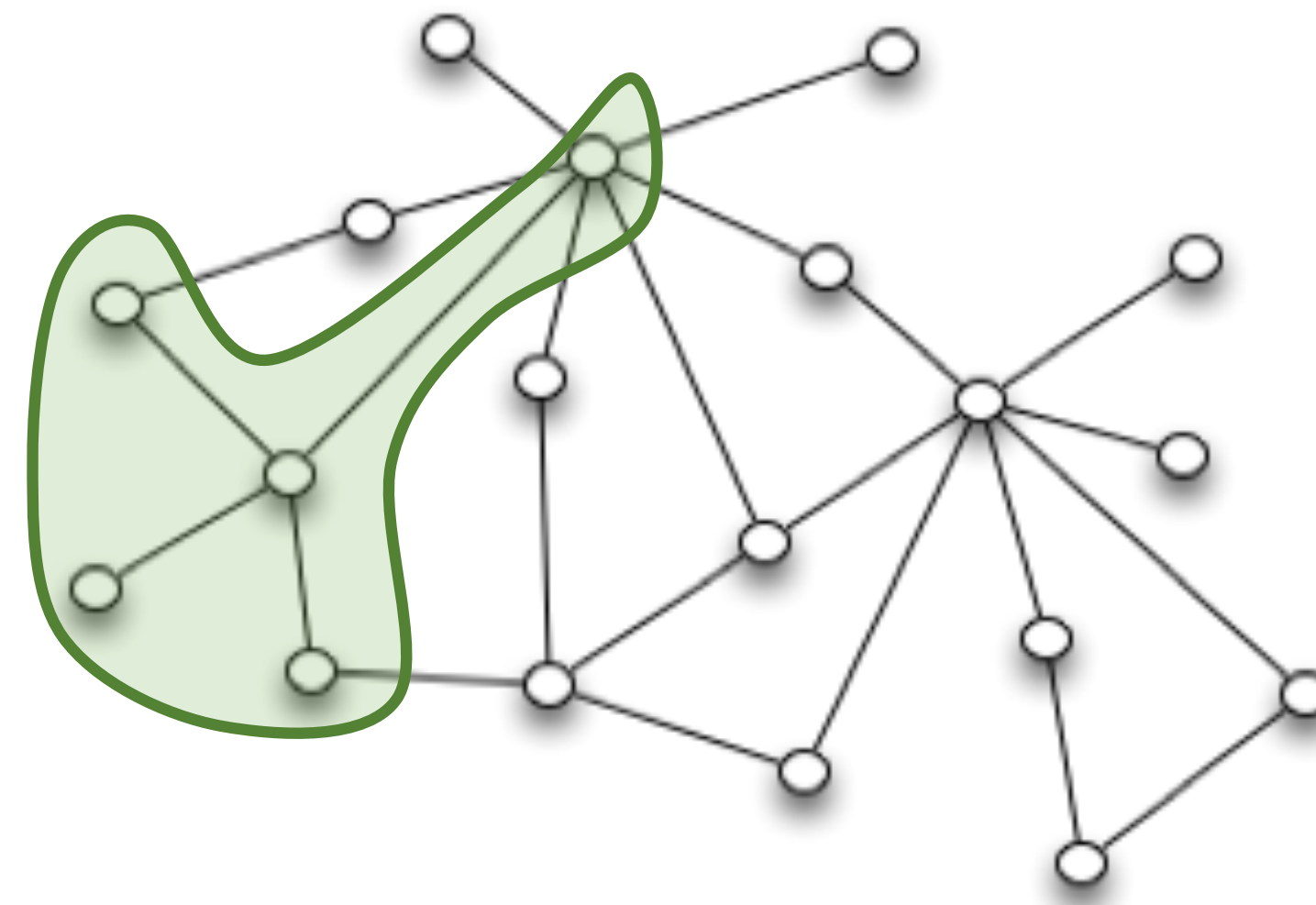
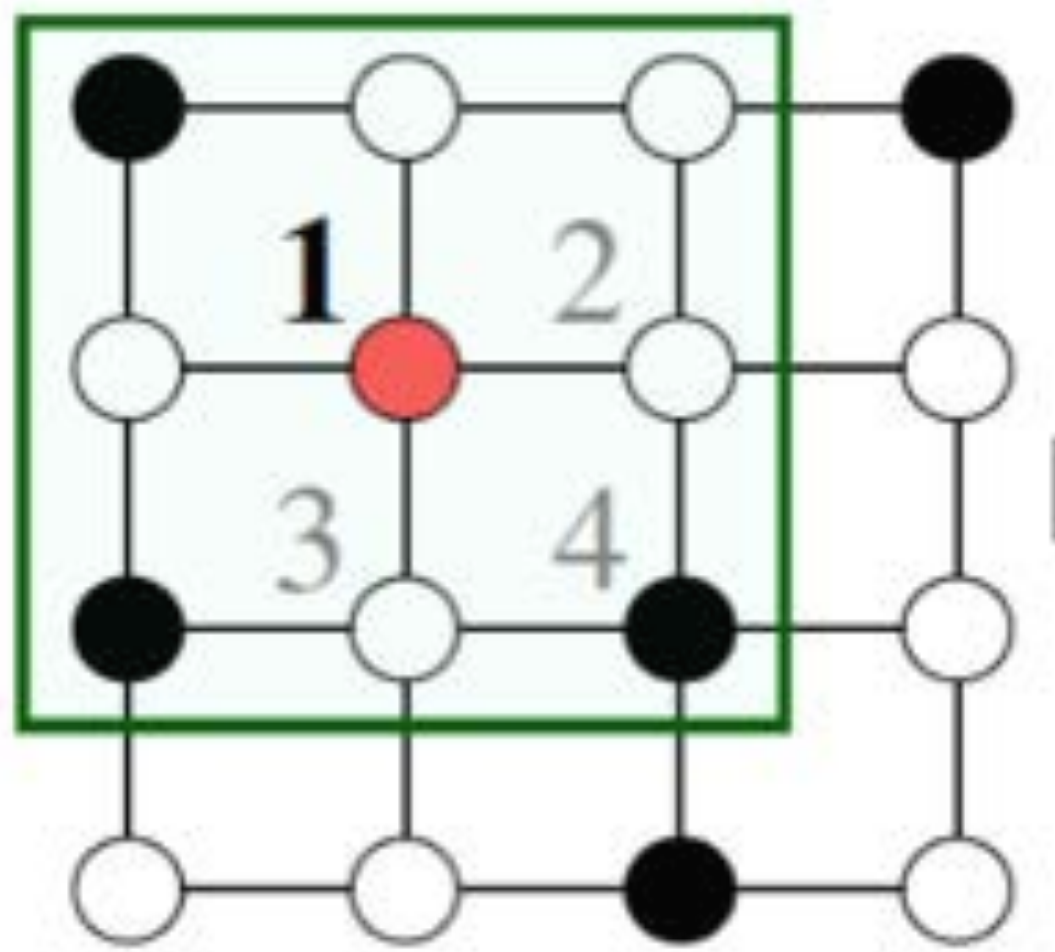
$$f(\mathbf{PAP}^\top, \mathbf{PX}) = f(\mathbf{A}, \mathbf{X})$$

$$f(\mathbf{PAP}^\top, \mathbf{PX}) = \mathbf{P}f(\mathbf{A}, \mathbf{X})$$



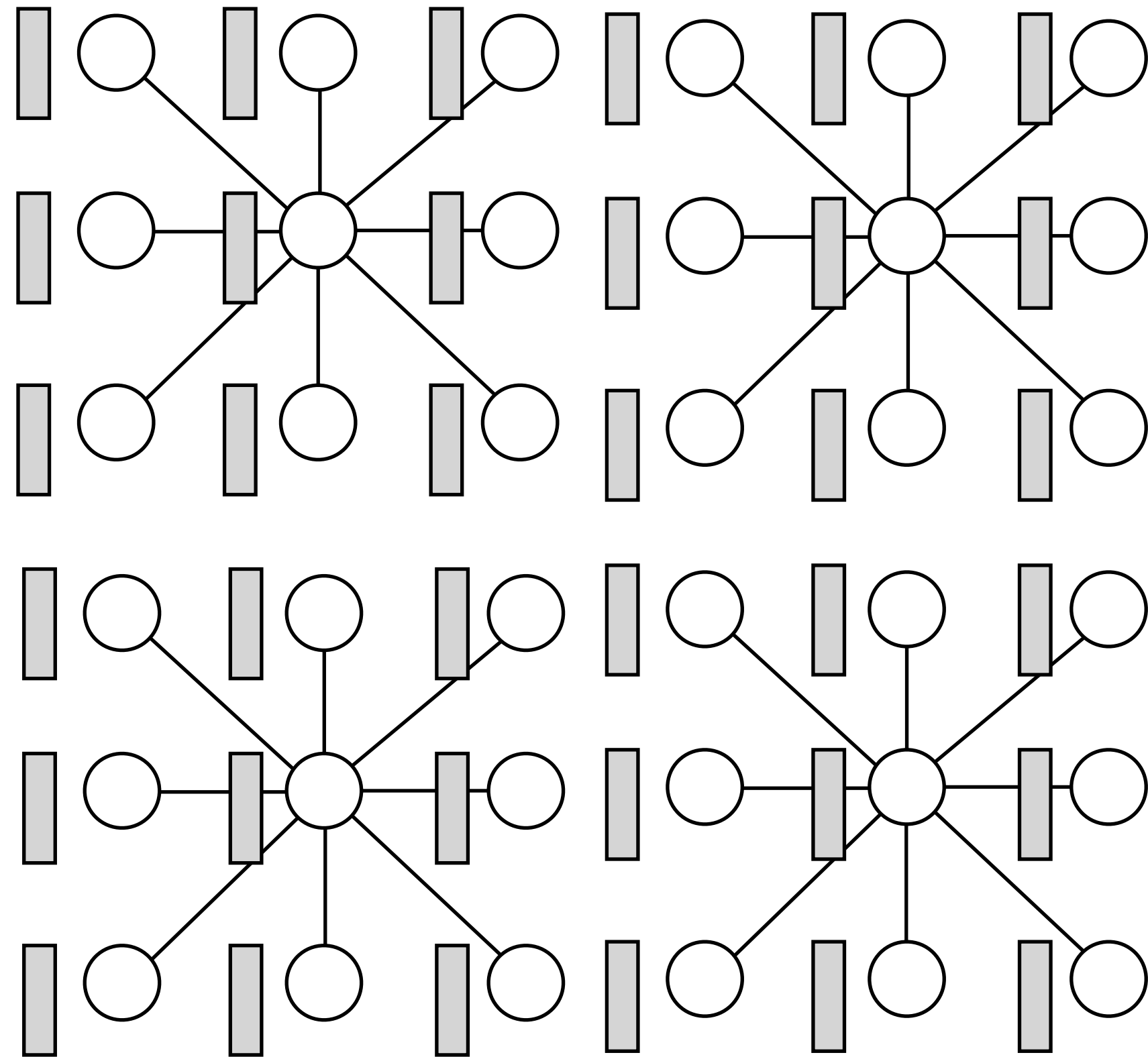
Idea 2: Images are like graphs...

- Convolution? (local operator “encodes” local neighborhoods)



- What is different in a graph?
- Commonalities: local operations, globalize through depth, weight sharing, input can have varying size

A CNN is a GNN over a grid graph



Review questions:

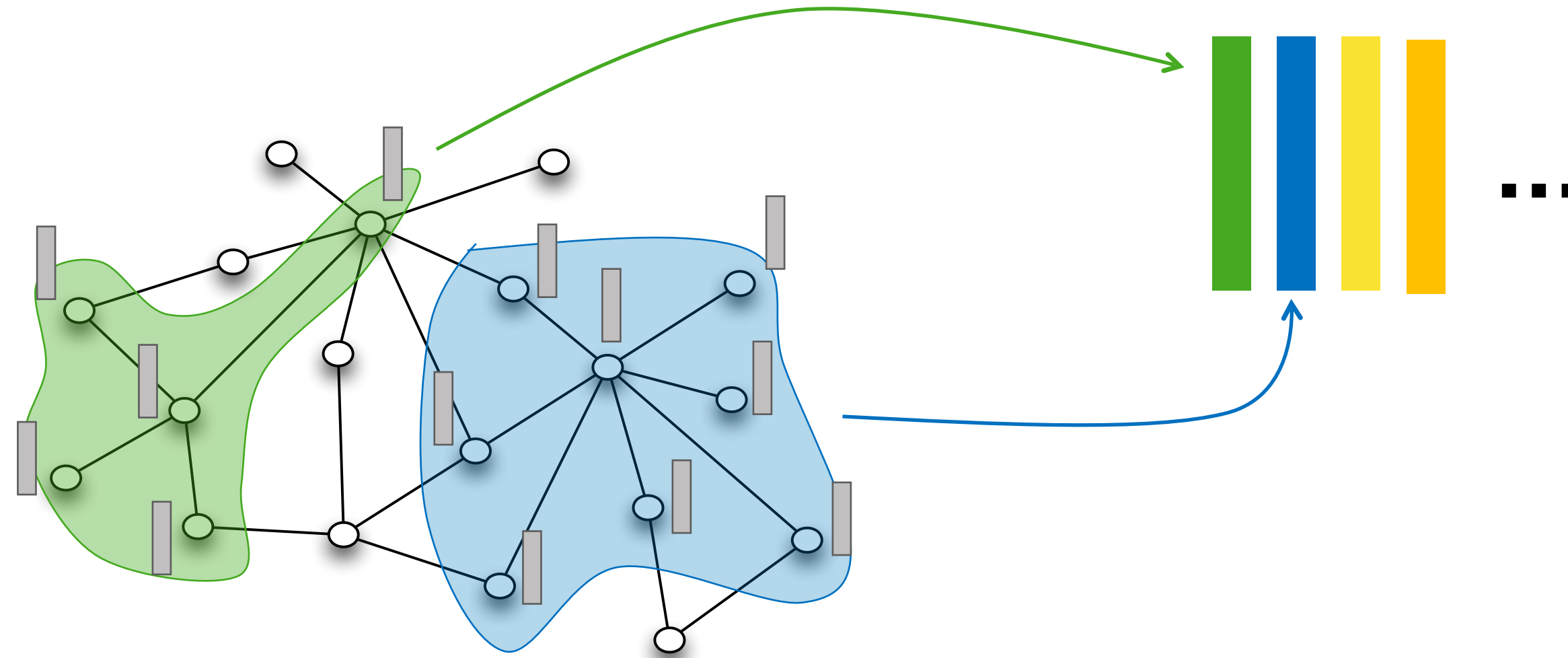
- What's the kernel size?
- What's the stride?

GNN's attribute vector per node == CNN's column of channels at each index in a feature map

Roadmap

- Learning tasks with graphs
- Message passing GNNs
- Approximation Power

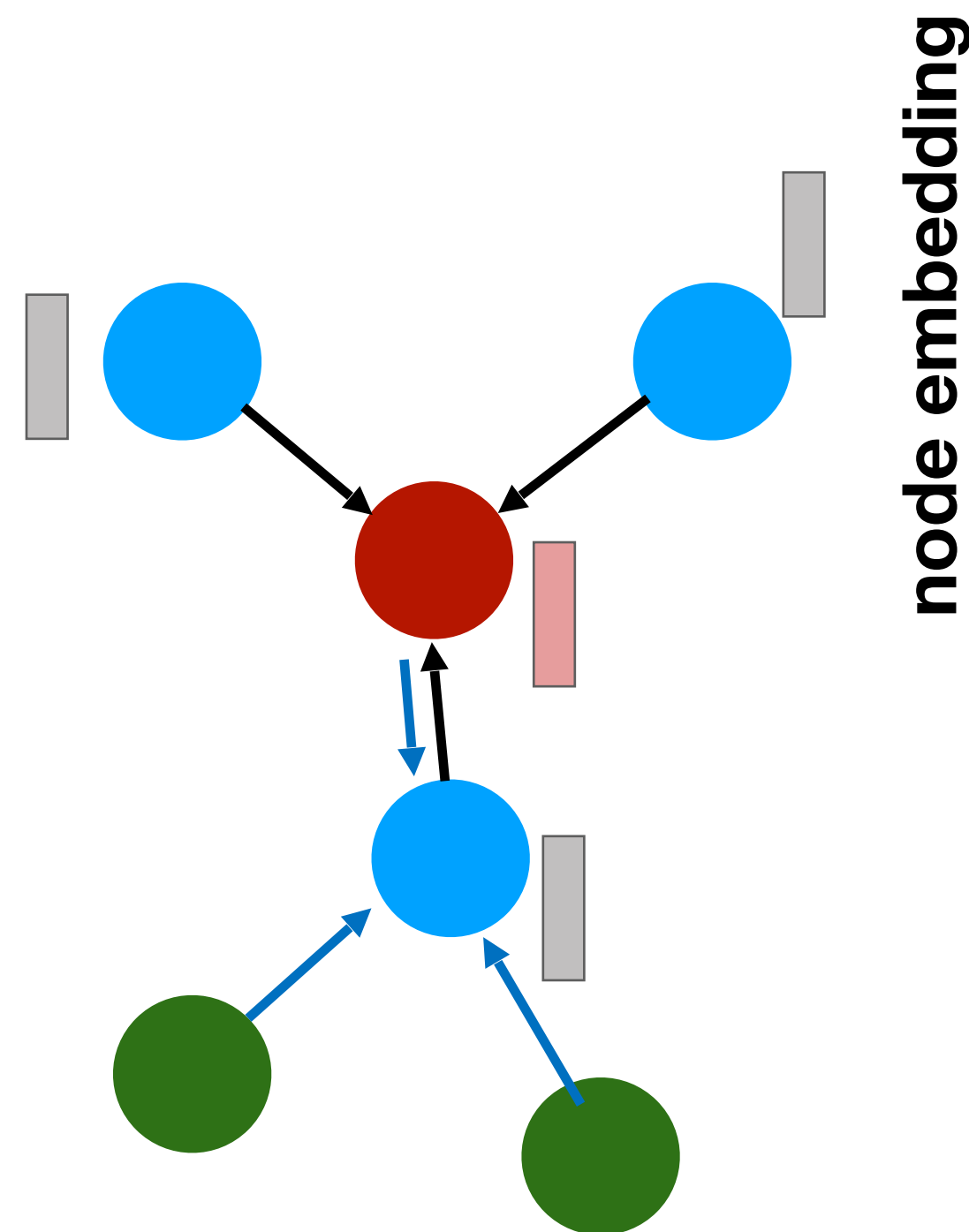
Graph neural networks



Idea:

1. Encode each node (based on message passing between nodes)
2. Aggregate set of node embeddings into a graph embedding

Encoding neighborhoods: general form



In each round k :

Aggregate information from neighbors

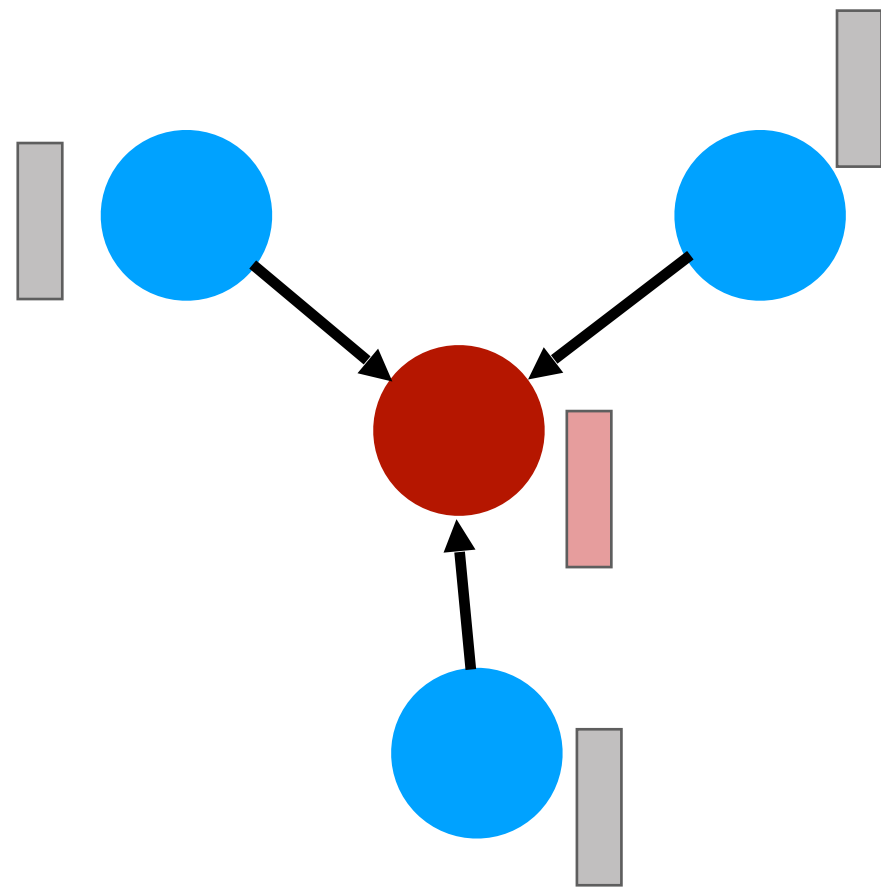
$$\mathbf{m}_{\mathcal{N}(v)}^{(k)} = \text{AGGREGATE}^{(k)}(\{\mathbf{h}_u^{(k-1)} : u \in \mathcal{N}(v)\})$$

*feature description
of node u in round $k-1$*

Update current node representation by incorporating messages from neighbors

$$\mathbf{h}_v^{(k)} = \text{UPDATE}^{(k)}(\mathbf{h}_v^{(k-1)}, \mathbf{m}_{\mathcal{N}(v)}^{(k)})$$

What are the aggregation functions?



$$\mathbf{m}_{\mathcal{N}(v)}^{(k)} = \text{AGGREGATE}^{(k)} \left(\{ \mathbf{h}_u^{(k-1)} : u \in \mathcal{N}(v) \} \right)$$

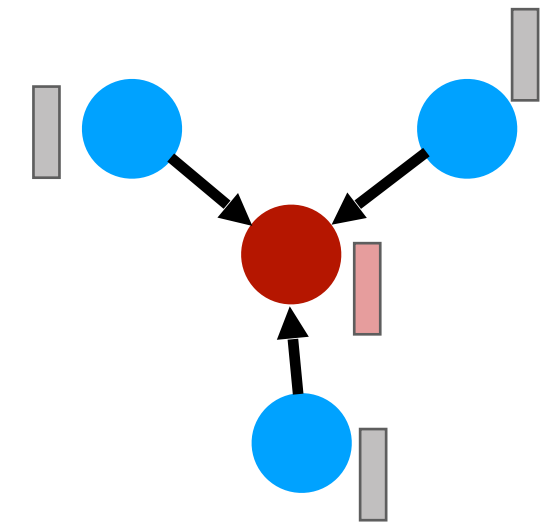
$$= \left(\bigoplus_{u \in \mathcal{N}(v)} \right) \psi^{(k)} \left(\mathbf{h}_u^{(k-1)}, \mathbf{h}_v^{(k-1)} \right)$$

$$\mathbf{h}_v^{(k)} = \text{UPDATE}^{(k)} \left(\mathbf{h}_v^{(k-1)}, \mathbf{m}_{\mathcal{N}(v)}^{(k)} \right)$$

$\mathcal{N}(v) = \{u \mid \exists(u, v) \in E(\mathcal{G})\}$ node's neighborhood

What are the aggregation functions?

Aggregate: permutation invariant, multi-set function



- Sum, average, ...
$$\mathbf{m}_{\mathcal{N}(v)} = \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u$$
$$\mathbf{m}_{\mathcal{N}(v)} = \sum_{u \in \mathcal{N}(v)} \frac{\mathbf{h}_u}{\sqrt{|\mathcal{N}(v)| |\mathcal{N}(u)|}}$$

(Merkwirth & Lengauer 2005, Scarselli et al 2009)

(Kipf & Welling 2016, Hamilton et al 2017)

- Min / max (coordinate-wise)

$$\mathbf{m}_{\mathcal{N}(v)} = \max \{ \mathbf{h}_u^{(k-1)} : u \in \mathcal{N}(v) \}$$

Can implement e.g. shortest path: $d_v^{(k)} = \min_{u \in \mathcal{N}(v)} d_u^{(k-1)} + \text{cost}(u, v)$

Learning a shortest path algorithm

Bellman-Ford

for $k = 1 \dots |S| - 1$:

for u in S :

$$d[k][u] = \min_v d[k-1][v] + \text{cost}(v, u)$$

GNN

for $k = 1 \dots \text{GNN iter}$:

for u in S :

$$h_u^{(k)} = \sum_v \text{MLP}(h_v^{(k-1)}, h_u^{(k-1)})$$



↑
sum or max pooling

Aggregation functions and updates

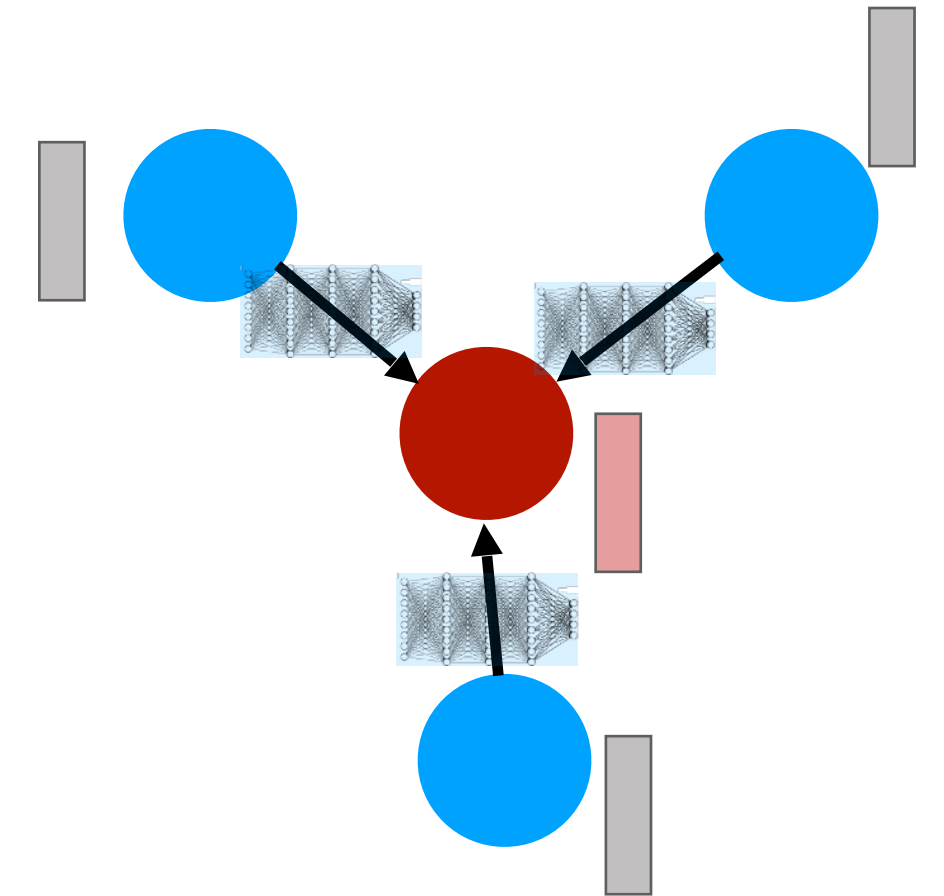
Aggregate: General form (*universal approximation of multi-set functions*): (Zaheer et al 2017, Qi et al 2017, Xu et al 2019)

$$\mathbf{m}_{\mathcal{N}(v)} = \text{MLP}_2 \left(\sum_{u \in \mathcal{N}(v)} \text{MLP}_1(\mathbf{h}_u, \mathbf{h}_v) \right)$$

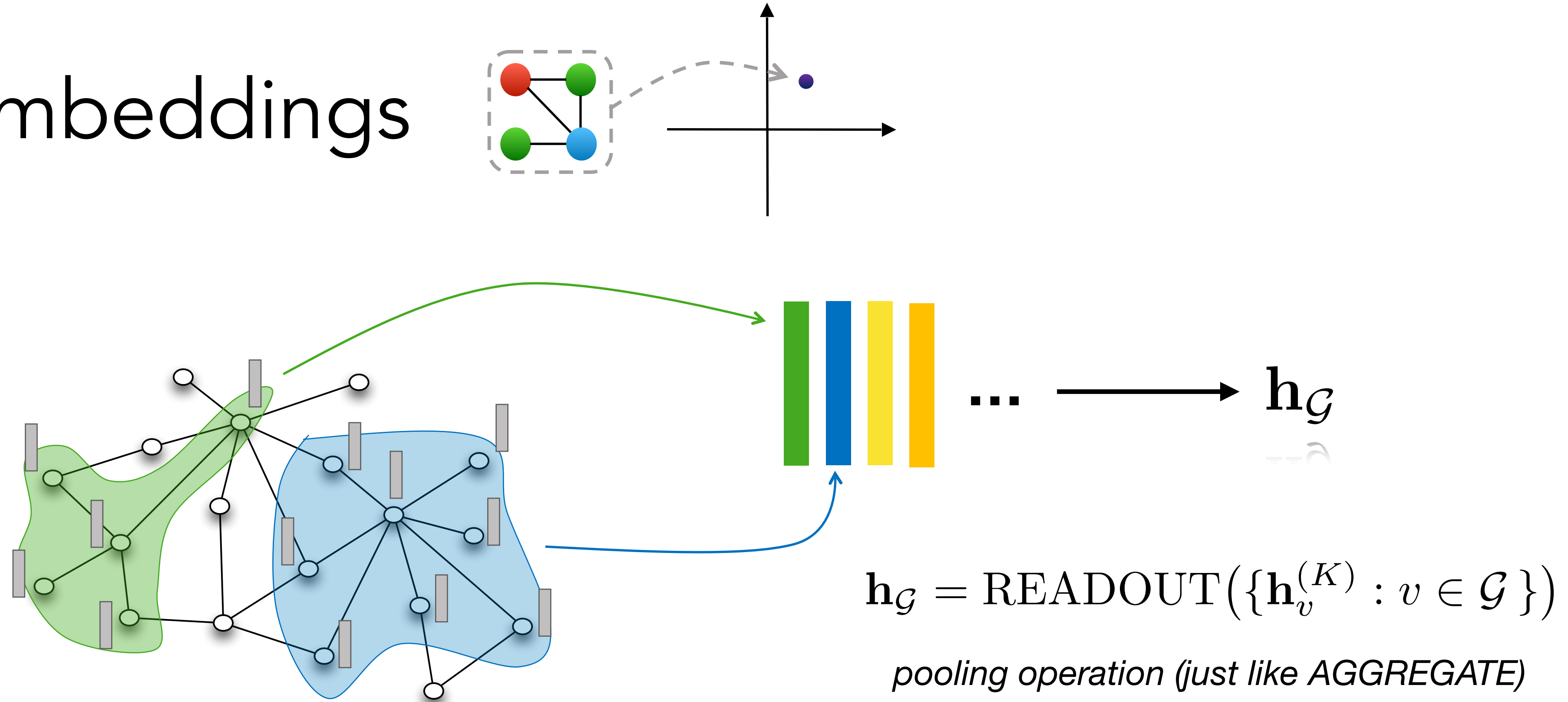
Learned aggregation function

Update: e.g.
$$\mathbf{h}_v^{(k)} = \sigma(\mathbf{W}_{\text{self}} \mathbf{h}_v^{(k-1)} + \mathbf{W}_{\text{neigh}} \mathbf{m}_{\mathcal{N}(v)}^{(k)} + b)$$

learned



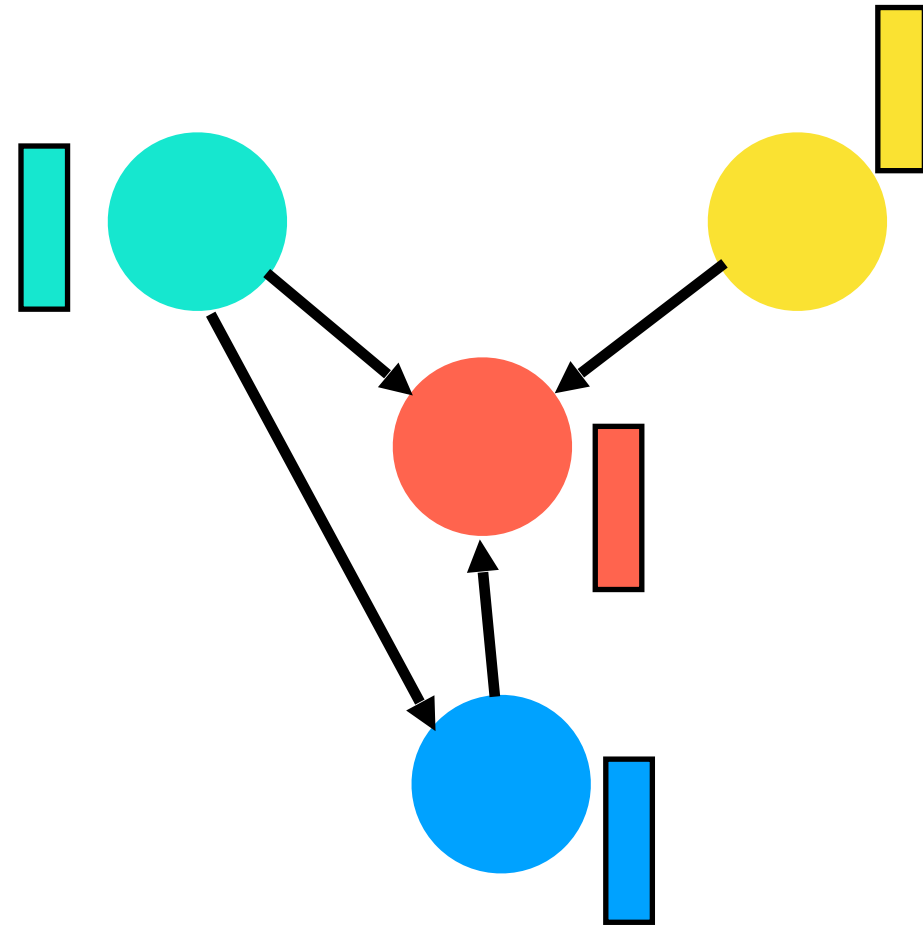
Graph embeddings



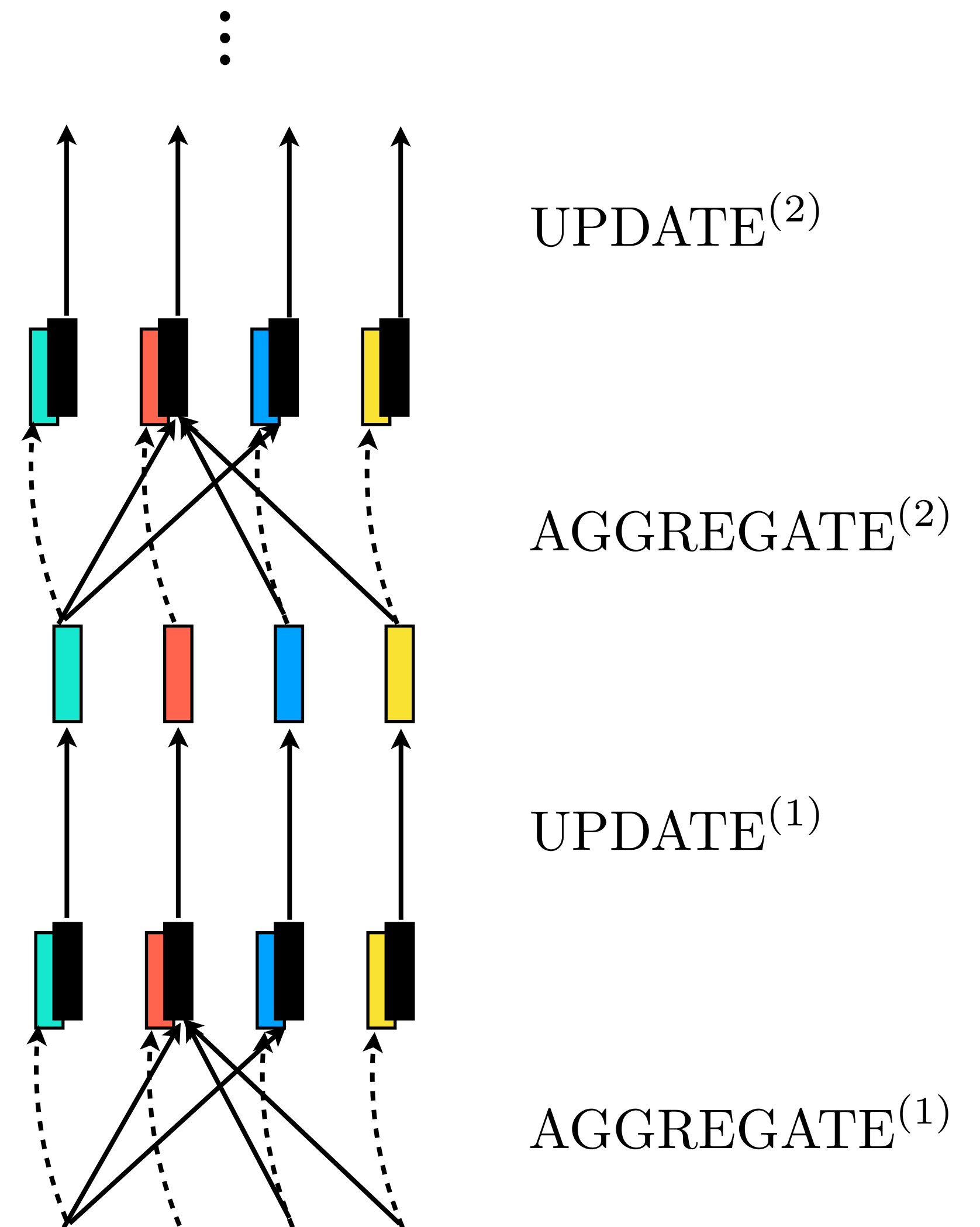
Idea:

1. Encode each node (based on message passing between nodes)
2. **Aggregate set of node embeddings into a graph embedding**

GNNs unrolled

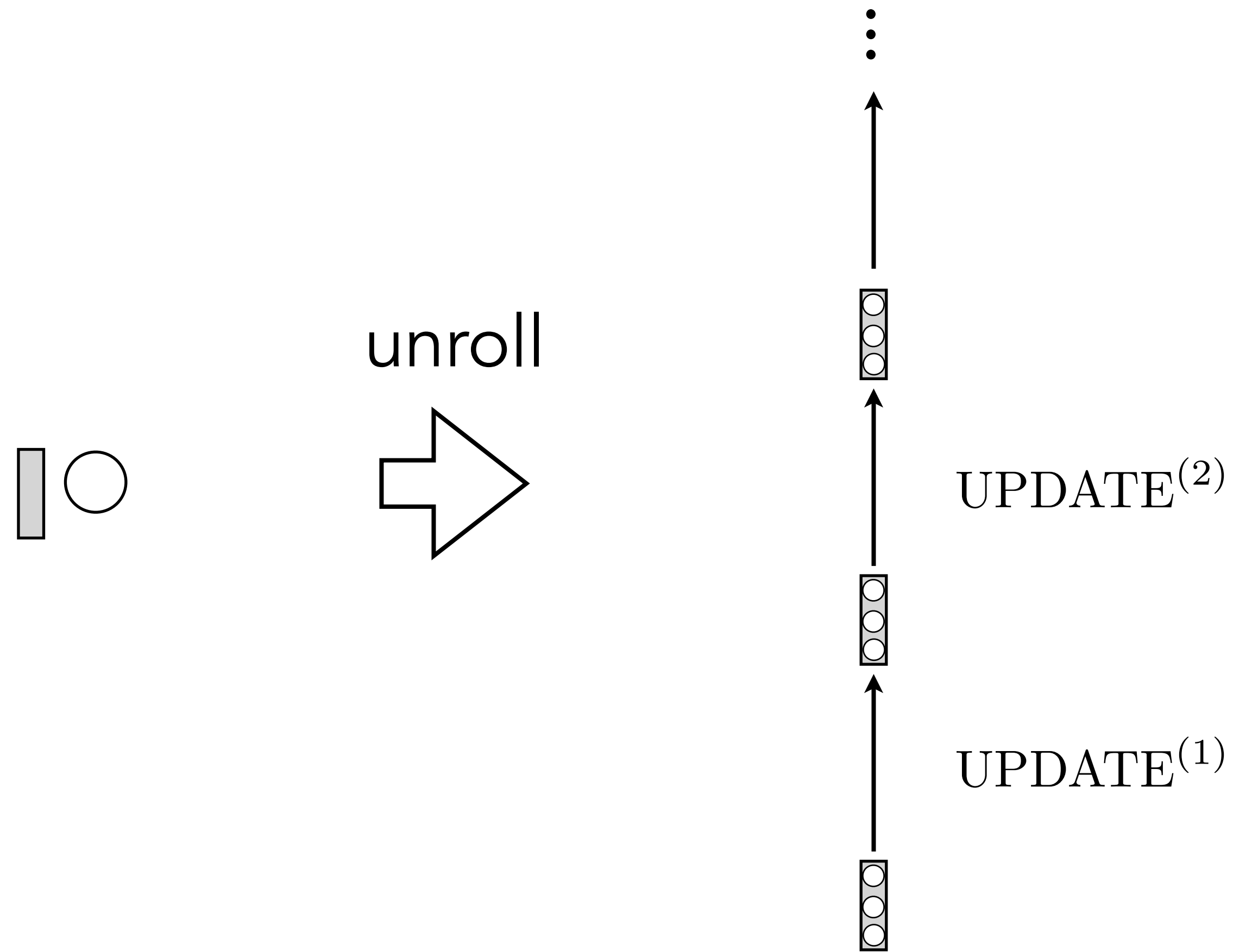


- Like an MLP, but nodes are vectors rather than scalars, edges are potentially complex functions (e.g., an edge can be an MLP)
- Each iteration of GNN message passing is a layer
 - AGGREGATE is akin to a linear layer
 - UPDATE is akin to a pointwise layer



What is the graph for an MLP?

An MLP is a GNN over single node



Update:

$$\mathbf{h}_v^{(k)} = \sigma(\mathbf{W}_{\text{self}} \mathbf{h}_v^{(k-1)} + \mathbf{W}_{\text{neigh}} \mathbf{m}_{\mathcal{N}(v)}^{(k)} + b)$$

Generalizations

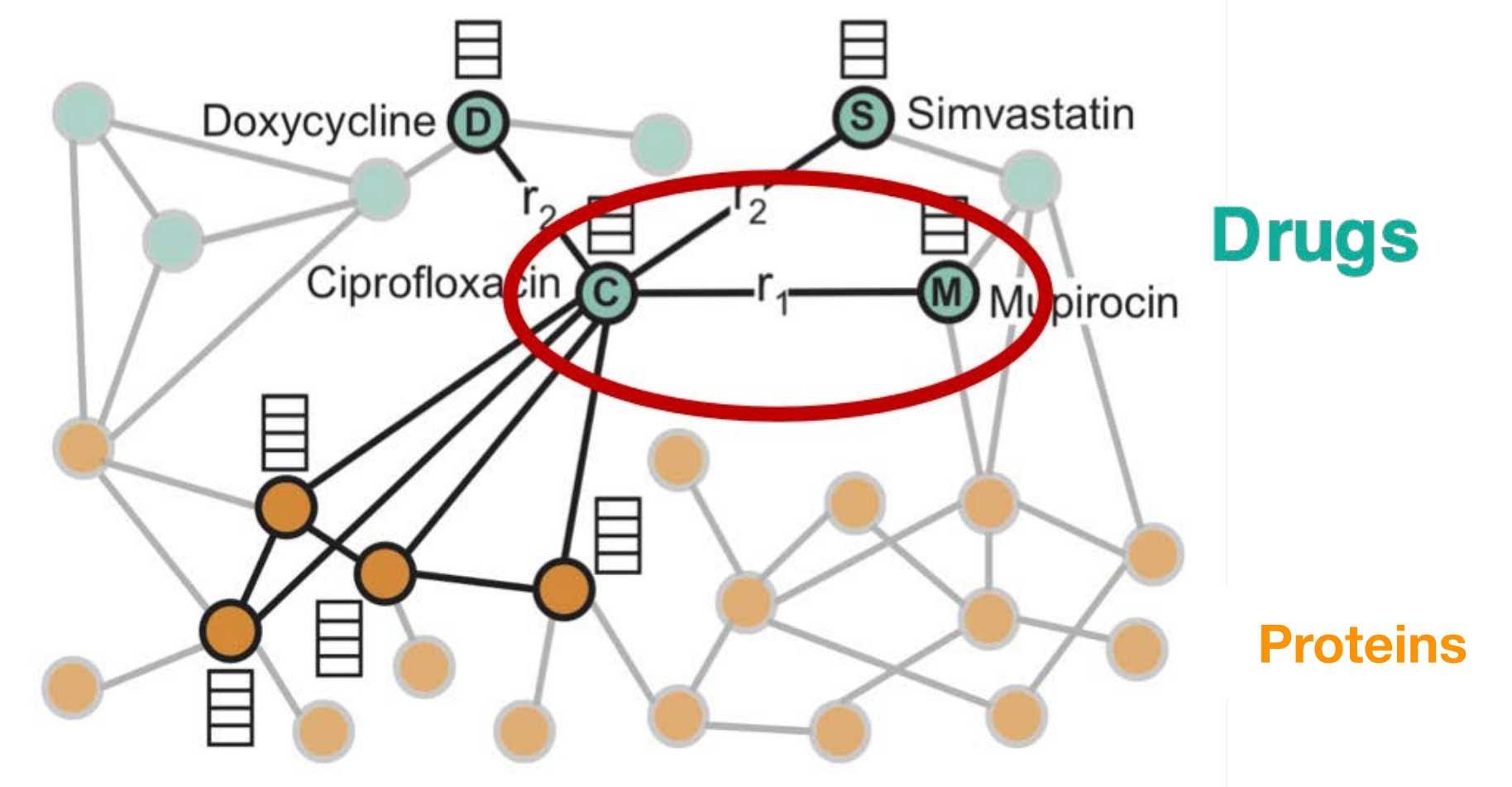
- Use **edge attributes** / features in aggregation
- **Multi-relational**: multiple “channels”
different aggregations for different types of edges

- **Attention** (Velickovic et al 2018):

$$\mathbf{m}_{\mathcal{N}(v)} = \sum_{u \in \mathcal{N}(v)} \alpha_{v,u} \mathbf{h}_u$$

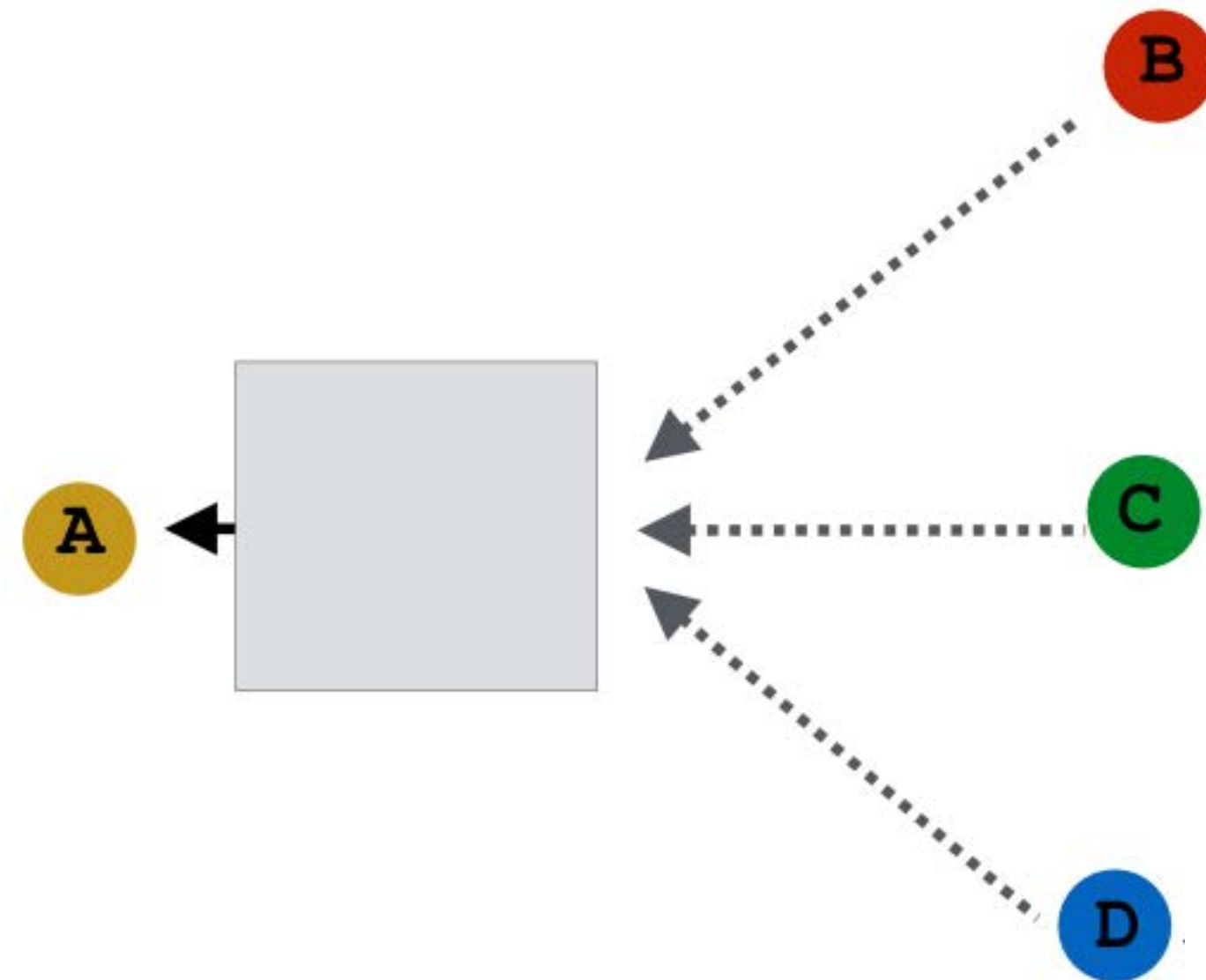
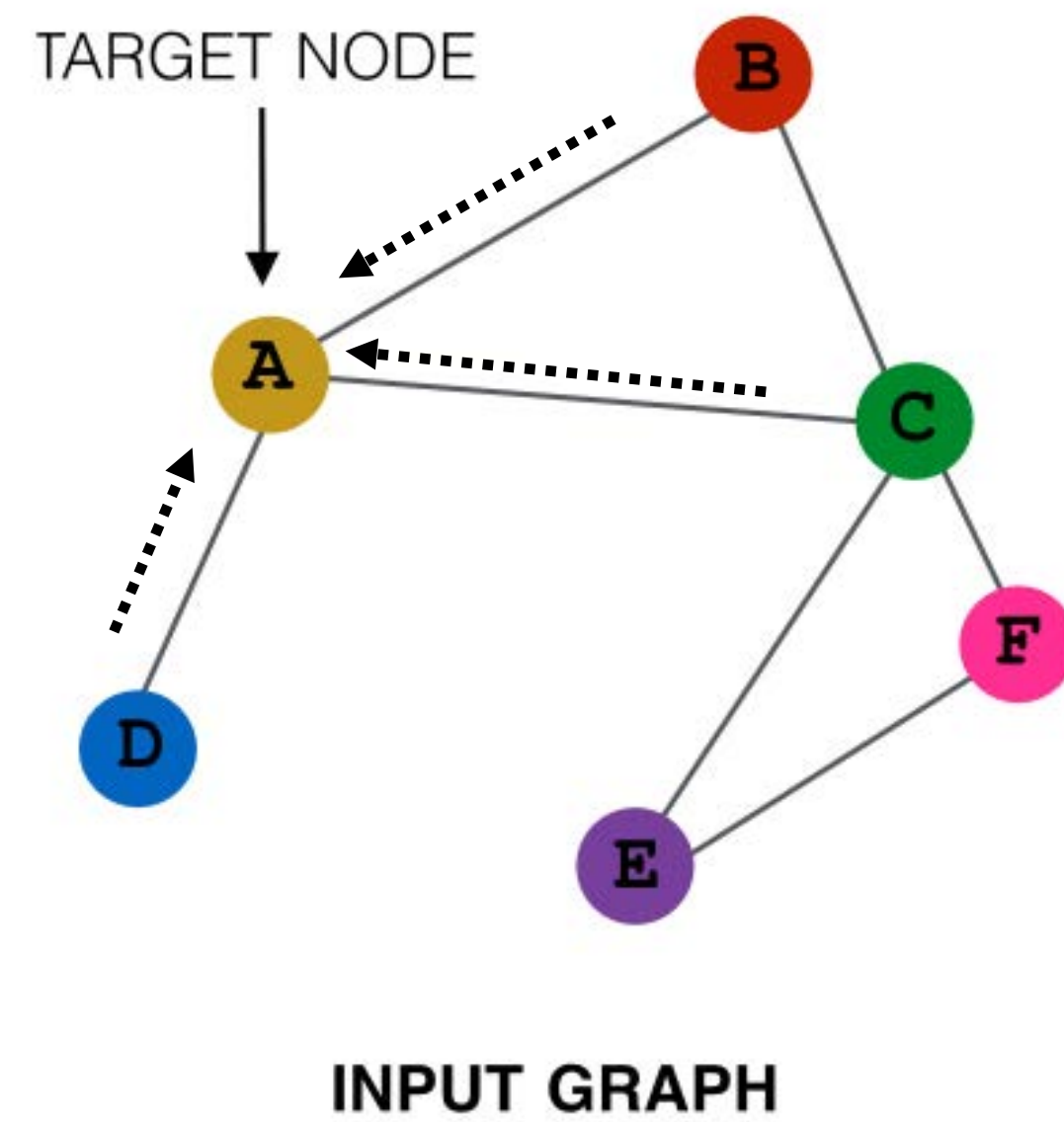
- Janossy pooling (Murphy et al 2018)
permutation-sensitive function averaged over permutations

$$\mathbf{m}_{\mathcal{N}(v)} = \sum_{u \in \mathcal{N}(v)} \text{MLP}^{(k)}(\mathbf{h}_u^{(k-1)}, \mathbf{h}_v^{(k-1)}, \mathbf{w}_{uv})$$



(Zitnik et al, 2018)

Node embeddings: tree view

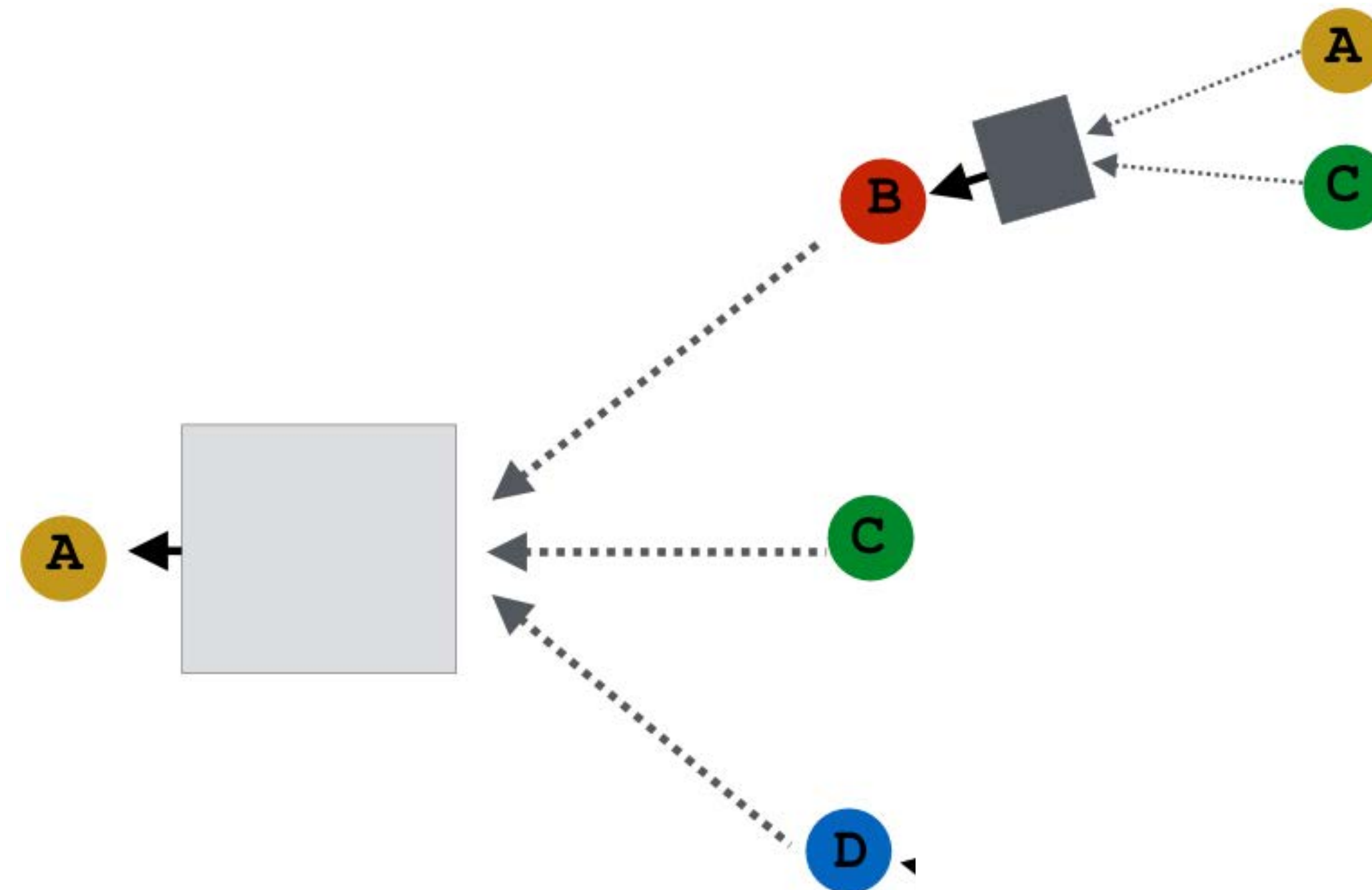
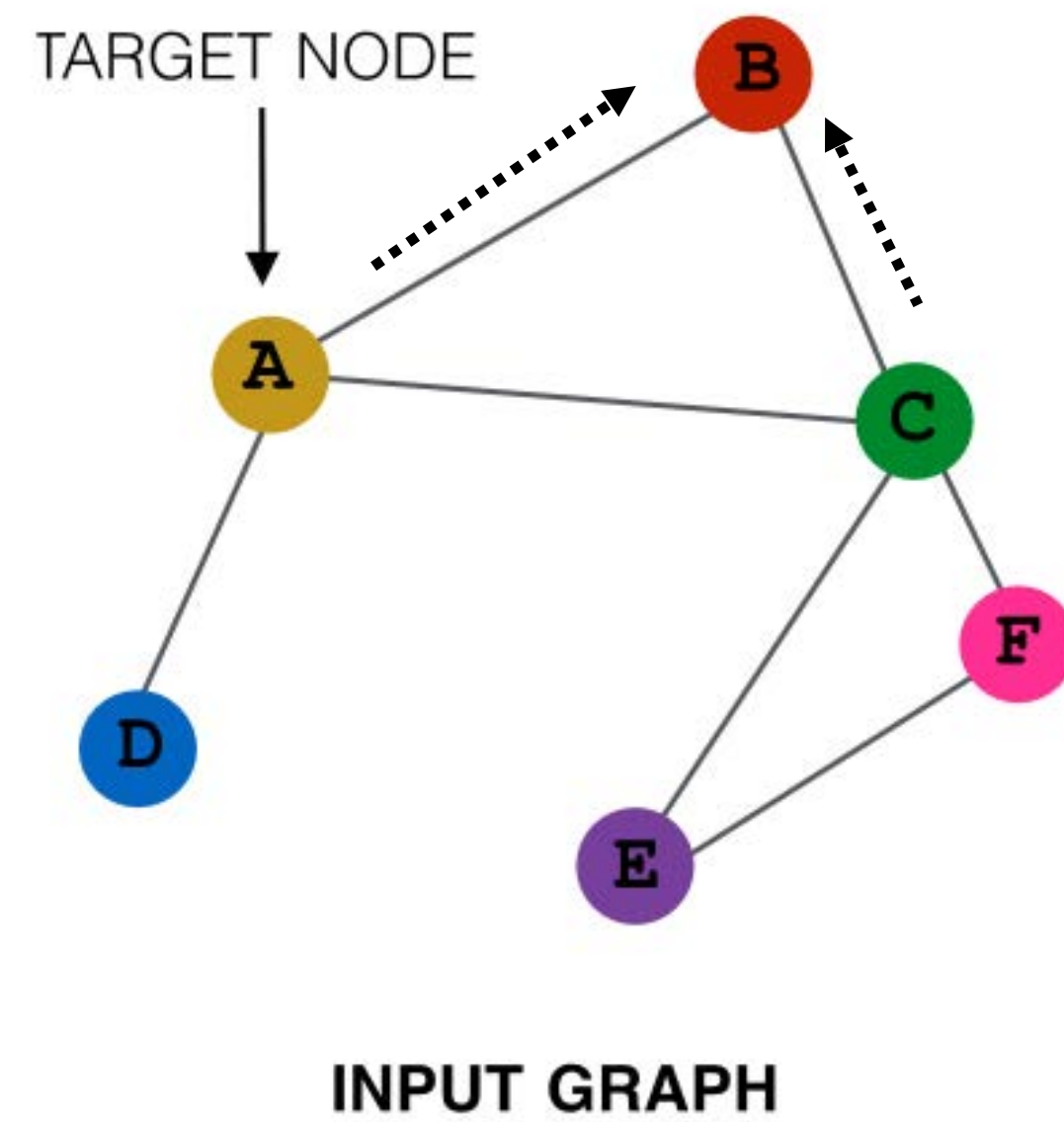


grey boxes: aggregation functions that we learn

Illustration © J. Leskovec. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>

(illustrations: J. Leskovec)

Node embeddings: tree view

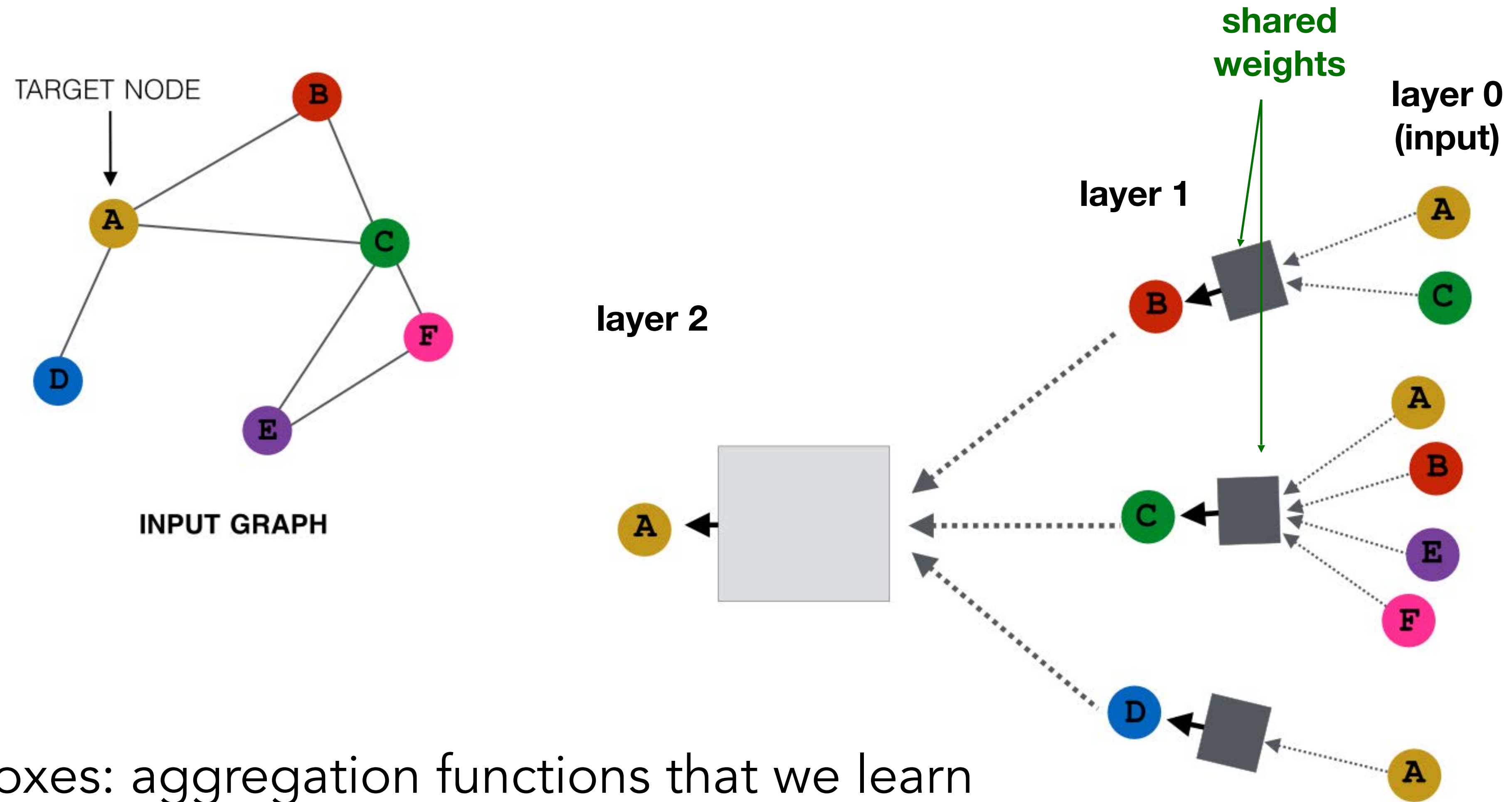


grey boxes: aggregation functions that we learn

Illustration © J. Leskovec. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>

(illustrations: J. Leskovec)

Node embeddings: tree view



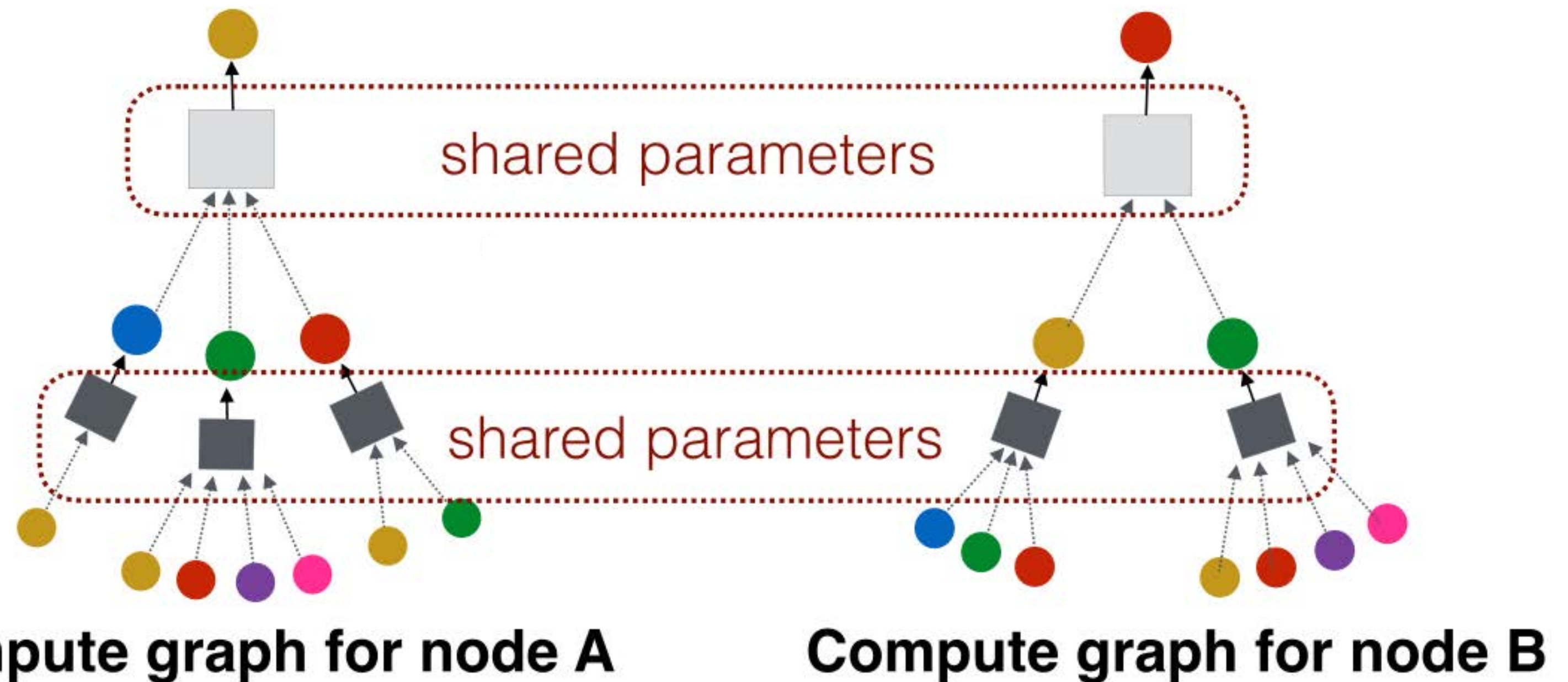
grey boxes: aggregation functions that we learn

Illustration © J. Leskovec. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>

(illustrations: J. Leskovec)

Weight sharing

- We use the same aggregation functions for all nodes.



- So we can generate encodings for **previously unseen nodes & graphs** too!
(dynamic graphs, different molecules, ...)

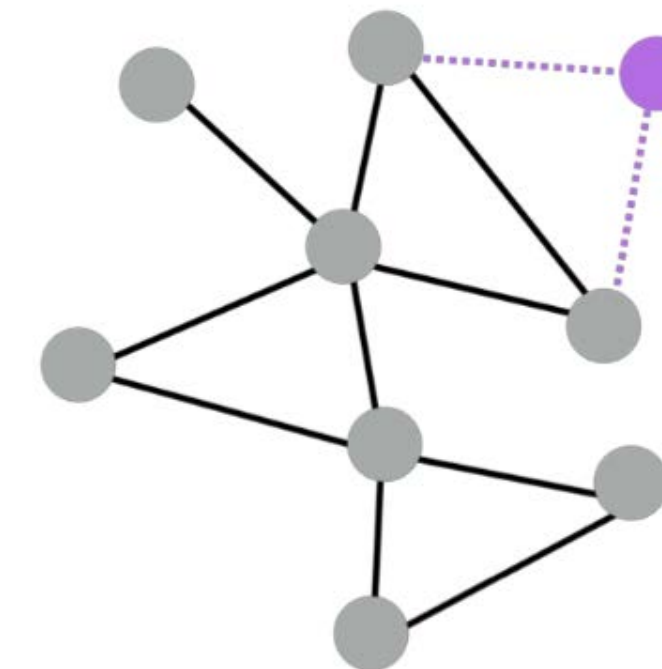
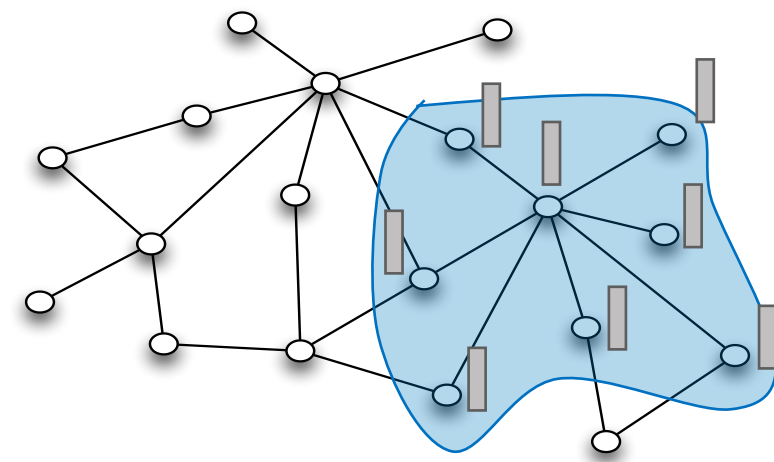


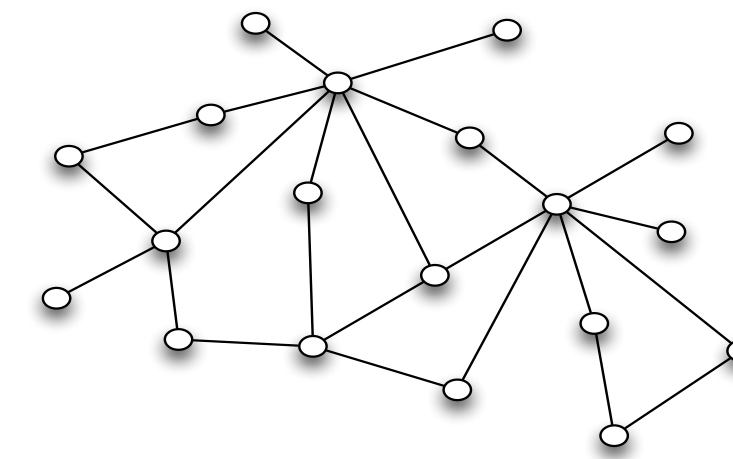
Illustration © J. Leskovec. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>

Training a GNN

- What is a data point?



{node, label} pairs

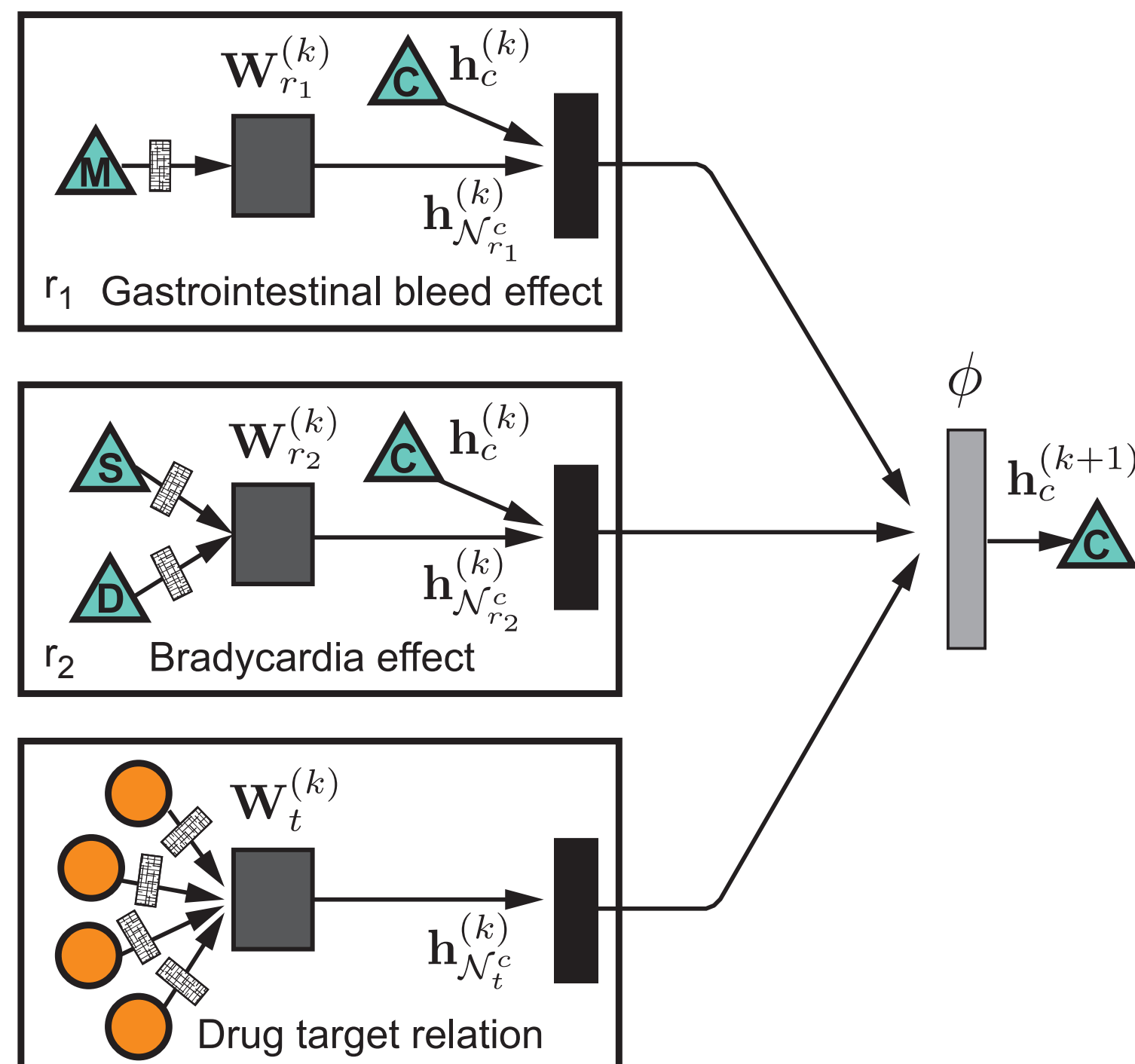
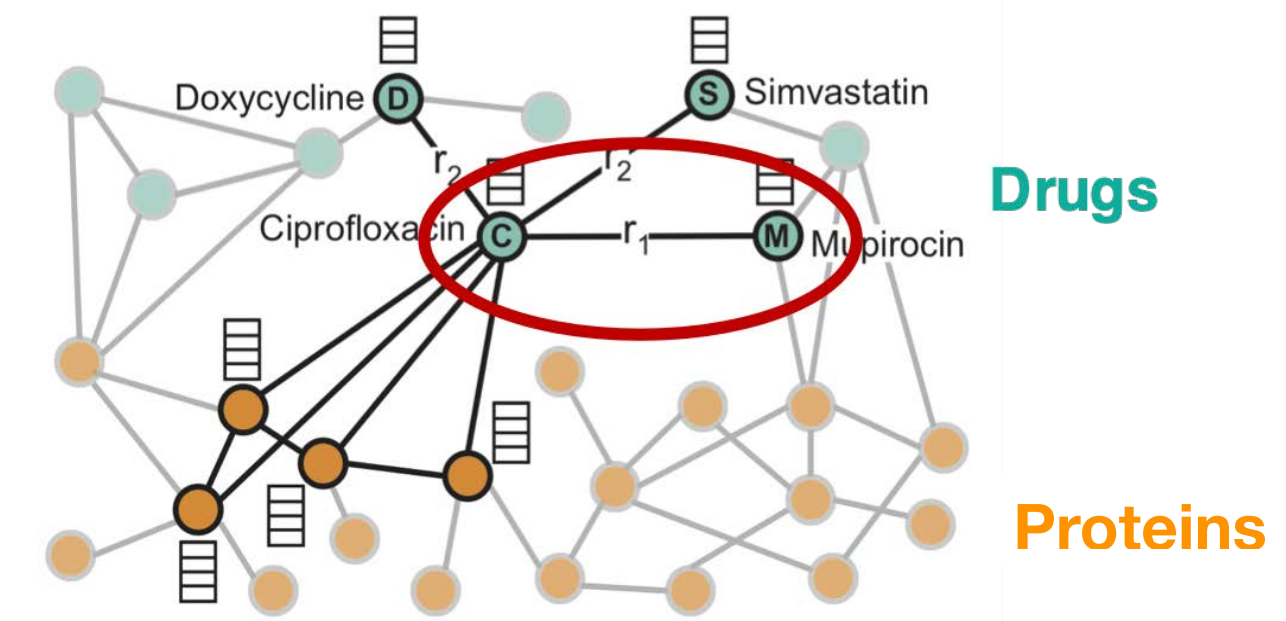


{graph, label} pairs

- What to specify?
 - Aggregate, updates and readout functions
 - Loss function on prediction
- Train with SGD

Example architecture 1: polypharmacy

- Different types of edges: drug-drug, drug-protein, protein-protein



$$\mathbf{h}_v^{(k+1)} = \text{ReLU} \left(\sum_r \left(\sum_{u \in \mathcal{N}_r(v)} c^{vu} \mathbf{W}_r^{(k)} \mathbf{h}_u^{(k)} + c_r^v \mathbf{h}_v^{(k)} \right) \right)$$

*separate aggregation per edge type r ,
then sum them up*

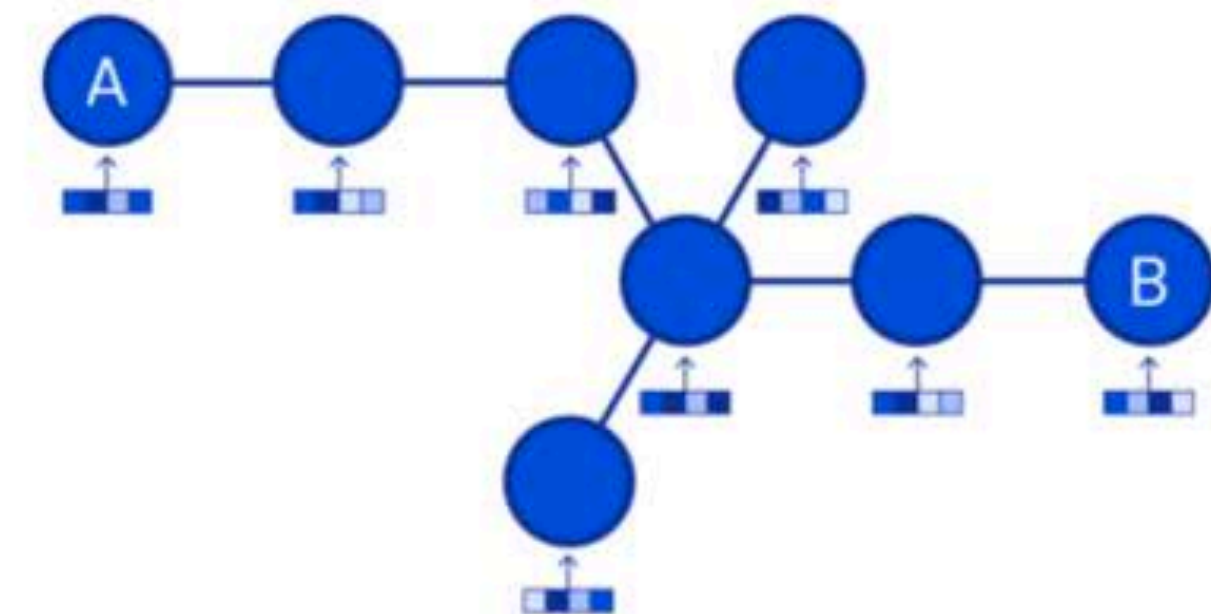
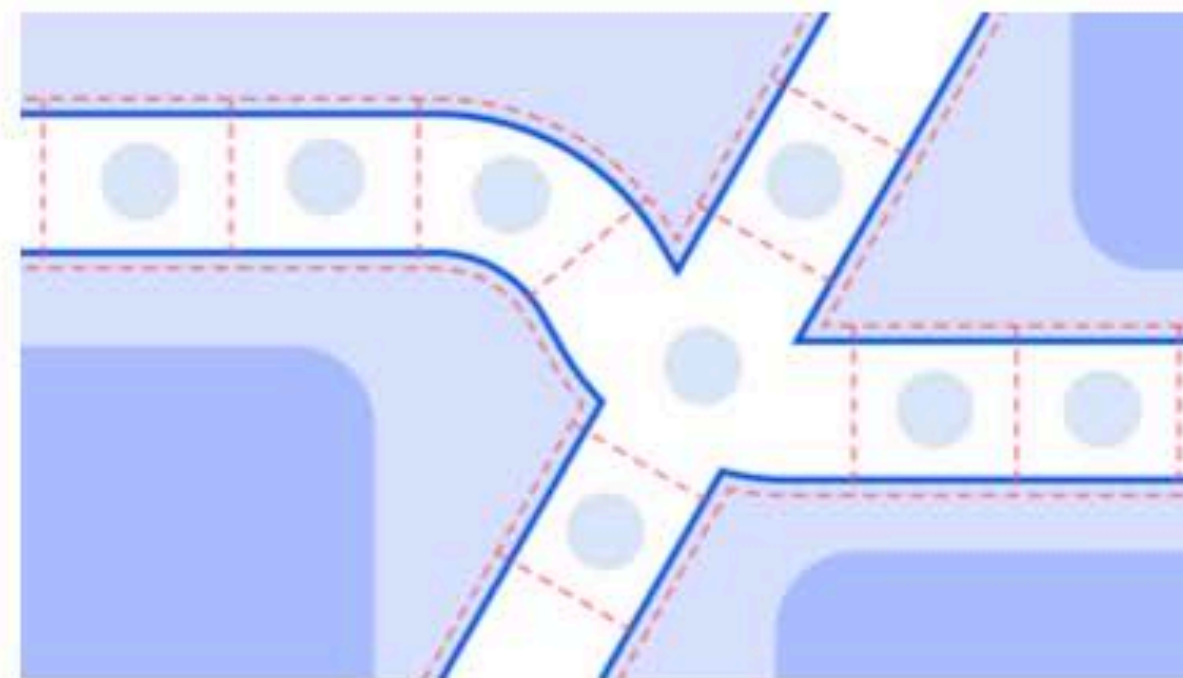
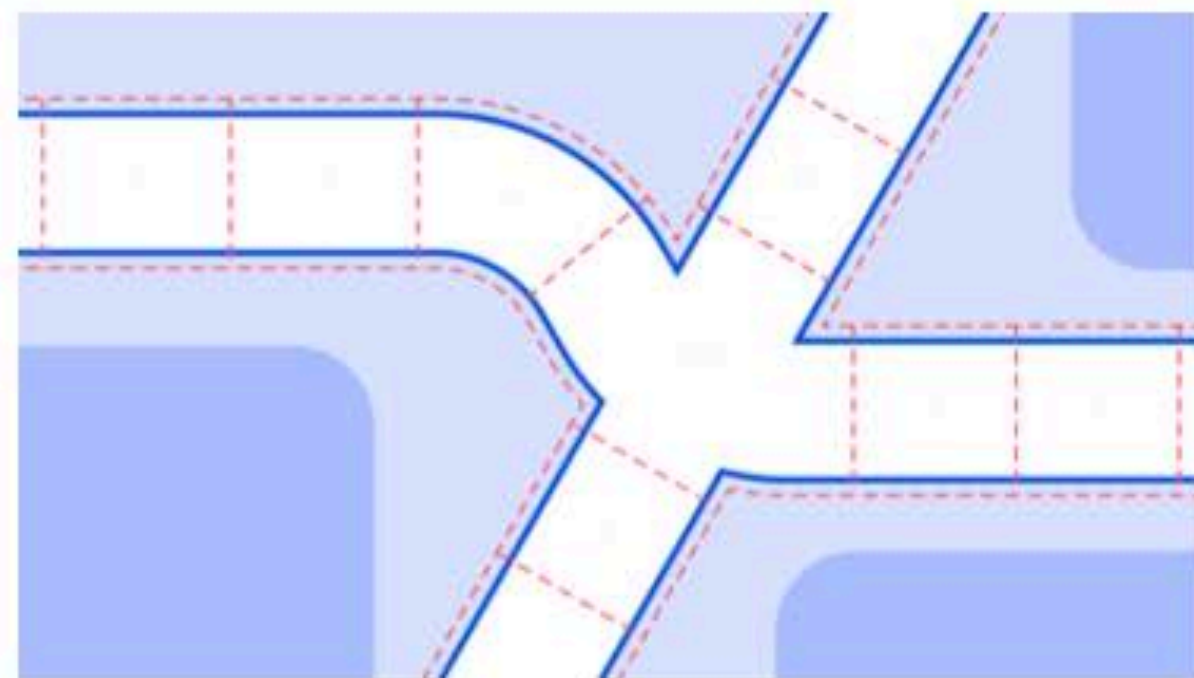
© Zitnik, et al. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>

Modeling Polypharmacy Side Effects with Graph Convolutional Networks

Marinka Zitnik¹, Monica Agrawal¹ and Jure Leskovec^{1,2,*}

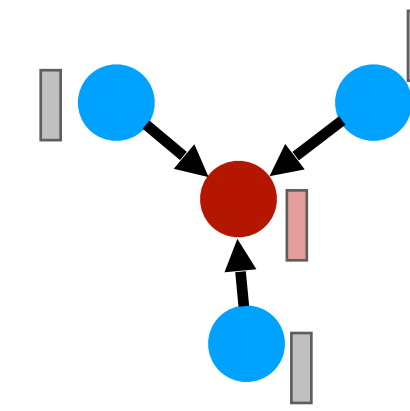
Example architecture 2: Google Maps

- segments v (50-100m), supersegments u (ca 20 segments);
input features: historical travel times, road type, traffic pattern at prediction time
- 3 aggregation/update operations: $\mathbf{h}_v$'s (using adjacent edges $\mathbf{h}_e$, $\mathbf{h}_u$),
edges $\mathbf{h}_e$ (using adjacent segments $\mathbf{h}_v$ and $\mathbf{h}_u$),
 $\mathbf{h}_u$ (using segments $\mathbf{h}_v$ and edges $\mathbf{h}_e$ in supersegment)
- combination of pooling operations for each aggregation
- linear combination of losses per time horizon (segment travel time, super segment time, cumulative segment time, self-supervised / generative loss)



Many connections

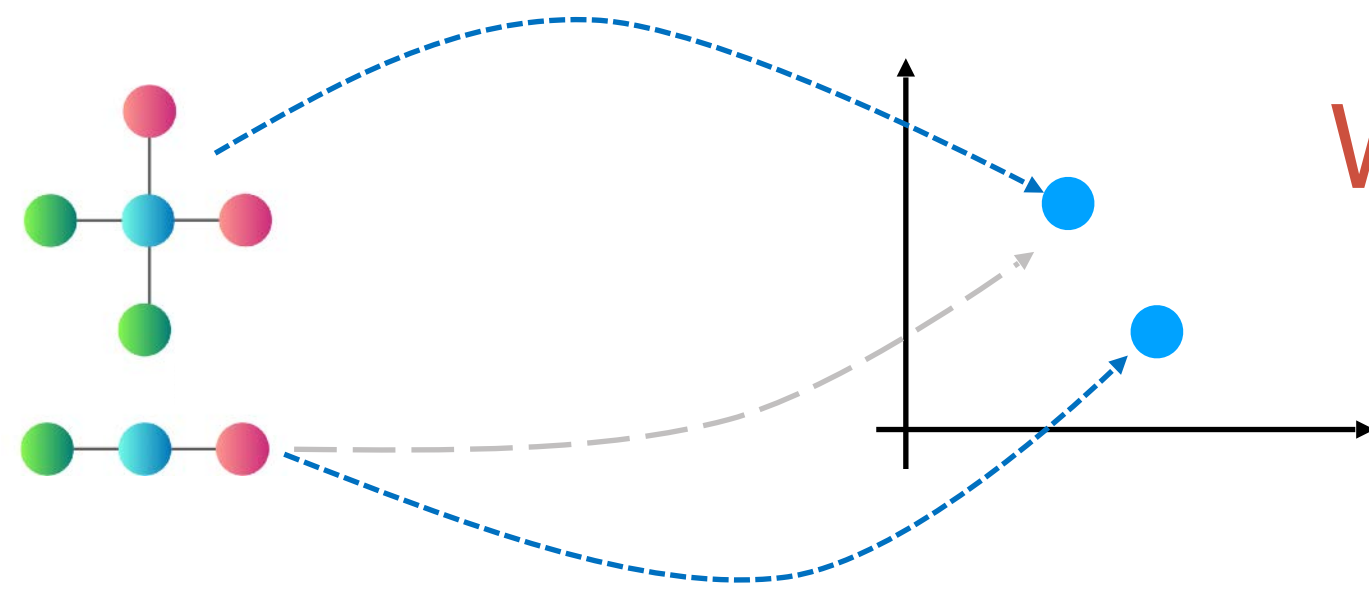
- **Graph signal processing and convolutions**
- **Inference in graphical models** *(Dai et al 2016)*
 - Node embeddings = latent variables
 - Given node features and graph, infer latent variables
 - “Neural message passing”
- **Distributed / Local algorithms** *(Sato et al 2019, Loukas 2020)*
 - Bounds for detection, verification, computation with GNNs
- **Random walks** *(Xu et al 2018)*
 - Oversmoothing, graph structure and depth
 - (Adaptive) skip connections
- **Graph isomorphism testing** *(Morris et al 2019, Xu et al 2019)*



Roadmap

- Learning tasks with graphs
- Message passing GNNs
- Approximation Power

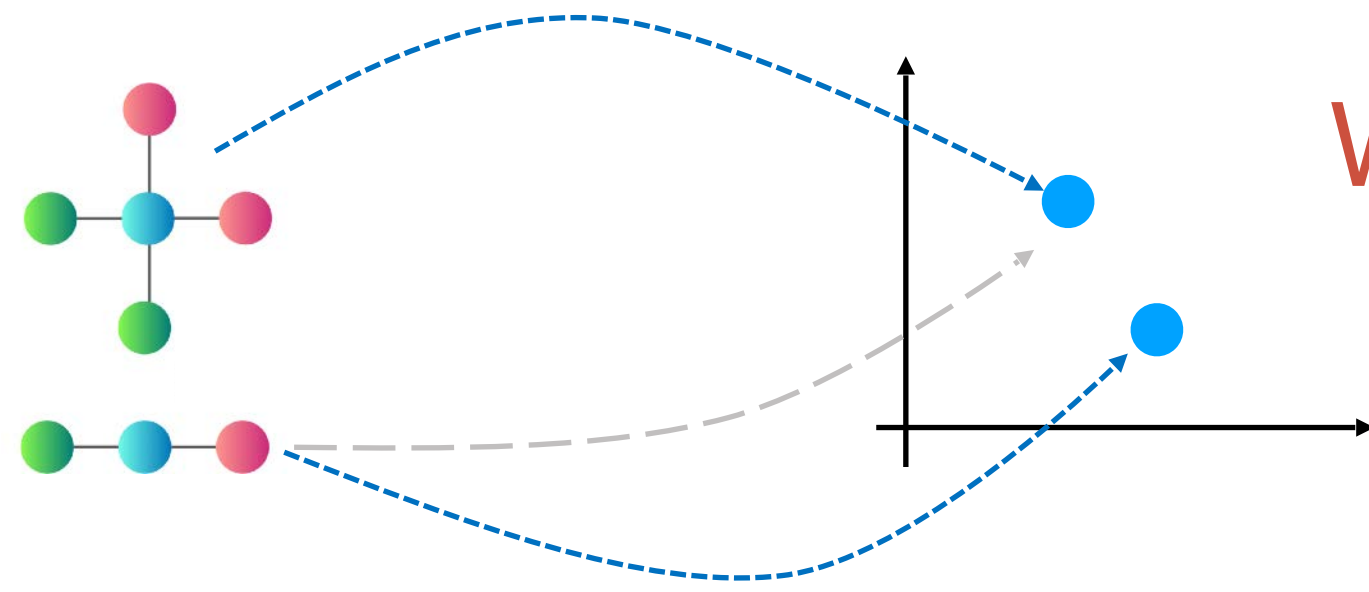
Which functions can GNNs approximate?



Which graphs can GNNs distinguish?

$$f(G) \neq f(G')?$$

Which functions can GNNs approximate?



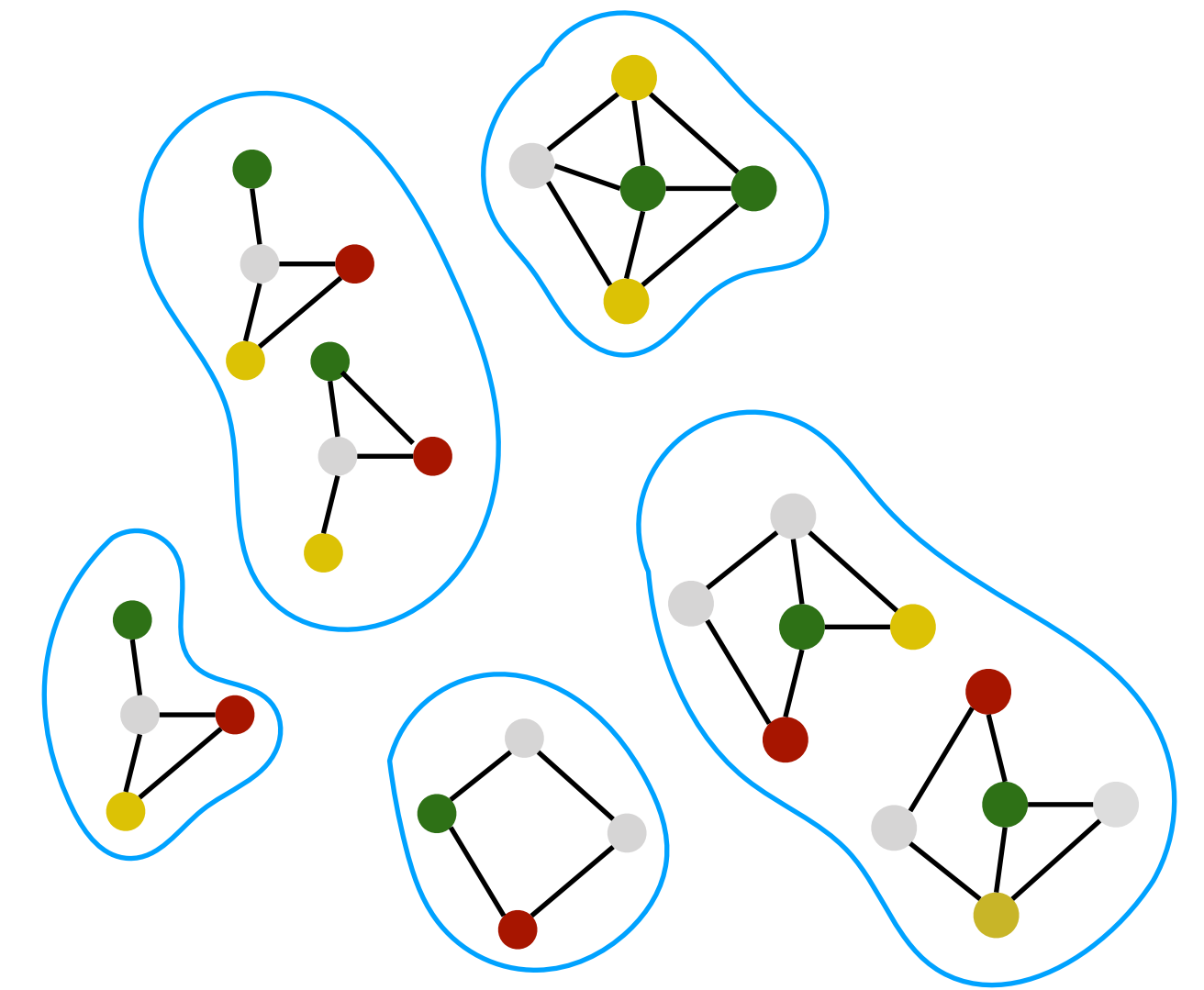
Which graphs can GNNs distinguish?

$$f(G) \neq f(G')?$$

Distinction implies function approximation
for node and graph predictions

(Symmetric Stone-Weierstrass theorem)

(Azizian & Lelarge 21, Chen-Villar-Chen-Bruna 19, Keriven & Peyré 19, Maron-Fetaya-Segol-Lipman 19)



$$(G, G') \in \rho(\mathcal{F}) \Leftrightarrow$$

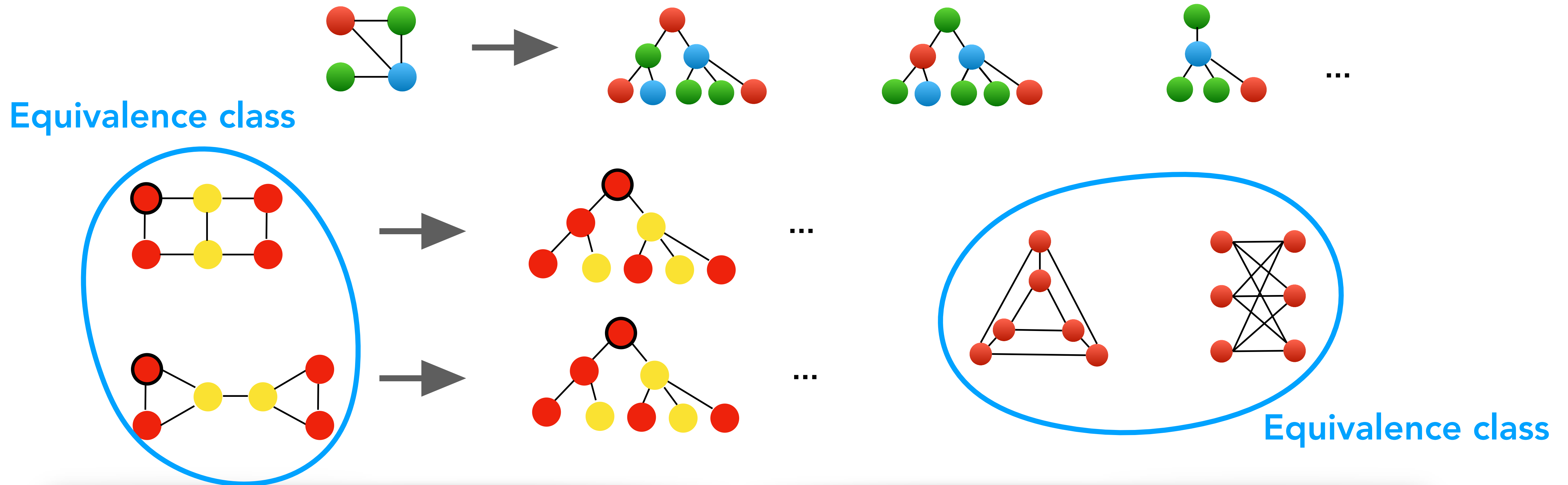
$$\forall F \in \mathcal{F}, F(G) = F(G')$$

Theorem.

If function H on a compact domain does not assign different labels to graphs in one equivalence class, then it can be approximated by message passing GNNs:

$$\forall \epsilon > 0, \exists F \in \mathcal{F}^{\text{GNN}} : \sup_{G \in K} \|H(G) - F(G)\| \leq \epsilon$$

Discriminative Power



Theorem *(Morris-Ritzert-Fey-Hamilton-Lenssen-Rattan-Grohe 19, Xu-Hu-Leskovec-Jegelka 19)*
Any GNN can at best distinguish the same graphs as the 1-dim WL algorithm.

Color refinement / Weisfeiler-Leman algorithm

(Morgan 65, Weisfeiler & Leman 68)



coloring $c^{(t)} : V(G) \rightarrow \Sigma$

$$c^{(t)}(v) = \text{Hash}\left(c^{(t-1)}(v), c^{(t-1)}(u) \mid u \in \mathcal{N}(v)\right)$$

isomorphism test: $\{c^{(t_\infty)}(v) \mid v \in V(G)\} \neq \{c^{(t_\infty)}(v') \mid v' \in V(G')\}?$

Color refinement / Weisfeiler-Leman algorithm

(Morgan 65, Weisfeiler & Leman 68)



coloring $c^{(t)} : V(G) \rightarrow \Sigma$

$$c^{(t)}(v) = \text{Hash}\left(c^{(t-1)}(v), c^{(t-1)}(u) \mid u \in \mathcal{N}(v)\right)$$

vs GNN: $\mathbf{h}_v^{(t)} = f_{\text{Update}}\left(\mathbf{h}_v^{(t-1)}, f_{\text{Agg}}\left(\{\mathbf{h}_u^{(t-1)} \mid u \in \mathcal{N}(v)\}\right)\right)$

Color refinement / Weisfeiler-Leman algorithm

(Morgan 65, Weisfeiler & Leman 68)



coloring $c^{(t)} : V(G) \rightarrow \Sigma$

$$c^{(t)}(v) = \text{Hash}\left(c^{(t-1)}(v), c^{(t-1)}(u) \mid u \in \mathcal{N}(v)\right)$$

vs GNN: $\mathbf{h}_v^{(t)} = f_{\text{Update}}\left(\mathbf{h}_v^{(t-1)}, f_{\text{Agg}}\left(\{\mathbf{h}_u^{(t-1)} \mid u \in \mathcal{N}(v)\}\right)\right)$

Theorem (Morris-Ritzert-Fey-Hamilton-Lenssen-Rattan-Grohe 19, Xu-Hu-Leskovec-J 19)

Any GNN can at best distinguish the same graphs as the 1-dim WL algorithm.

For any n , there exists a GNN such that for any t , $c^{(t)} \equiv h^{(t)}$.

How could we ensure that the aggregation is injective?

- Any (multi-)set function can be represented with nonlinear functions g_1, g_2 as:

$$f_{\text{Agg}}(S) = g_1 \left(\sum_{\mathbf{h} \in S} g_2(\mathbf{h}) \right)$$

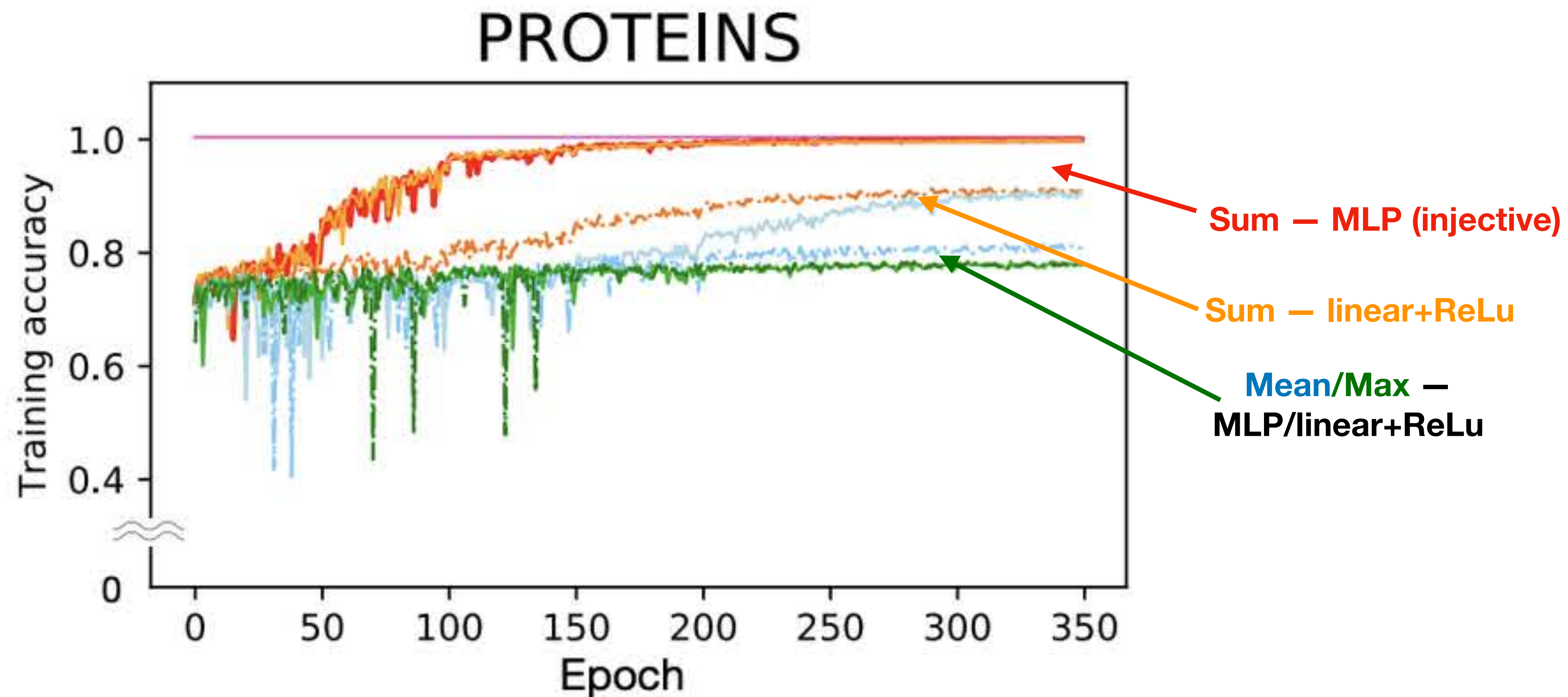
- We can universally approximate g_1 and g_2 by MLPs! (see a few slides ago)

$$\mathbf{m}_{\mathcal{N}(v)}^{(k)} = \text{MLP}_2 \sum_{u \in \mathcal{N}(v)} \text{MLP}_1(\mathbf{h}_u^{(k-1)})$$

Does this make a difference in practice?

- vary g and pooling operation

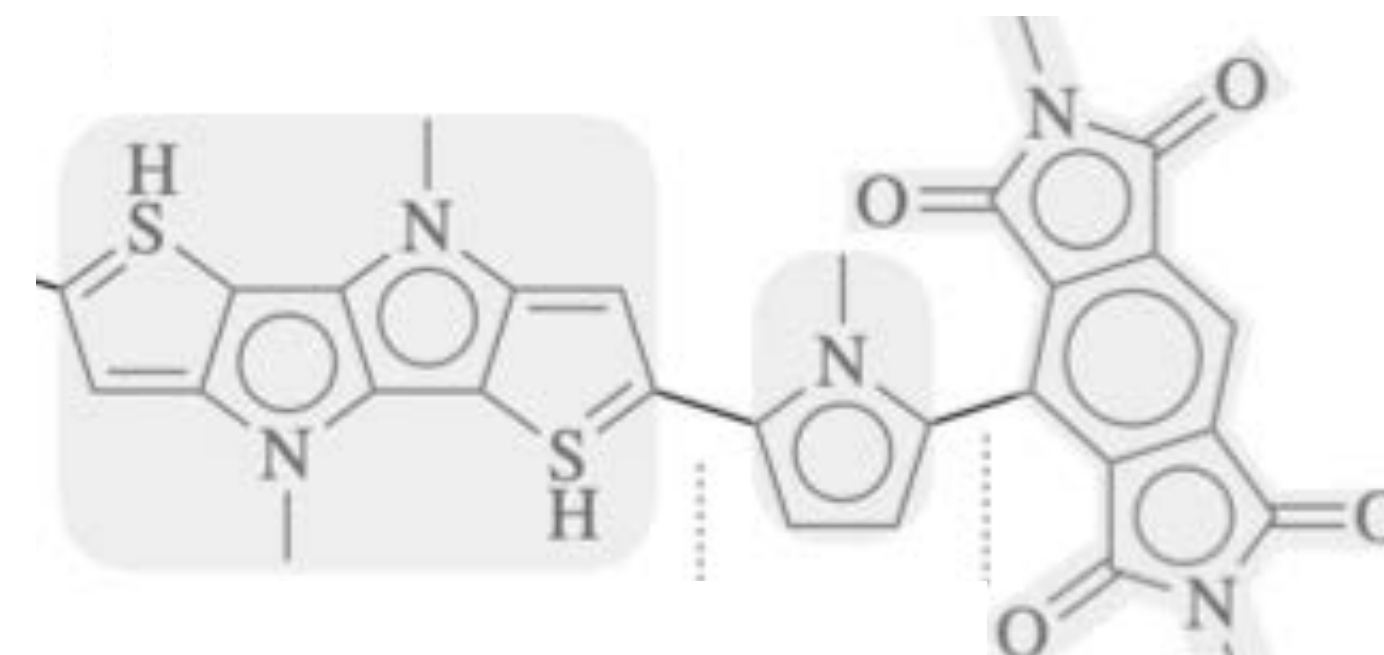
$$f_{\text{Agg}}(S) = g_1 \left(\sum_{\mathbf{h} \in S} g_2(\mathbf{h}) \right)$$



Learning structural graph properties

Can GNNs compute:

- the length of the shortest / longest cycle?
- diameter of the graph?
- the number of occurrences of a motif?



Lemma *(Garg et al 2020, Chen et al 2020)*

No! Message Passing GNNs (as discussed here) cannot compute these in general.

Improving discriminative power

- **Positional encodings**

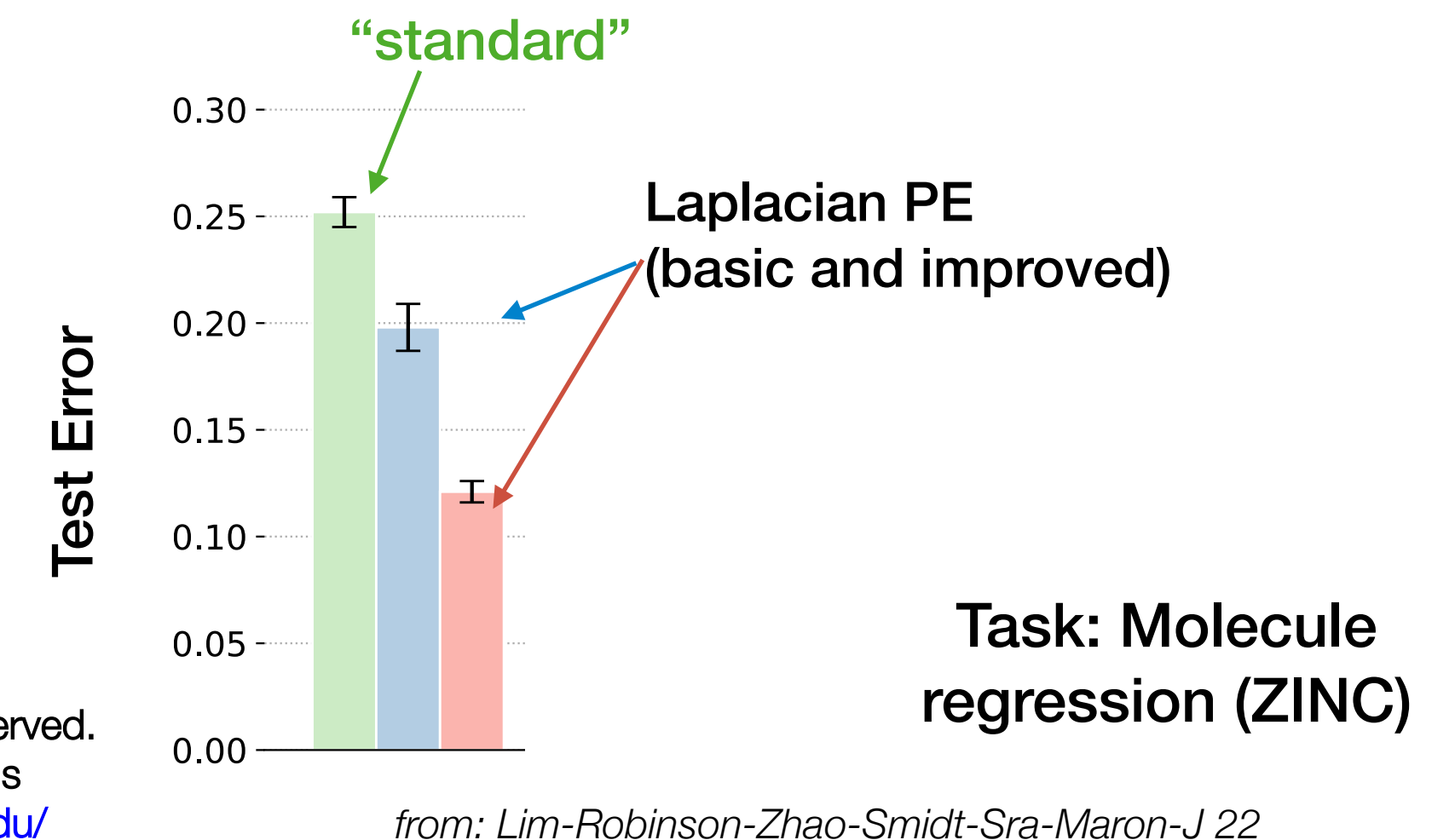
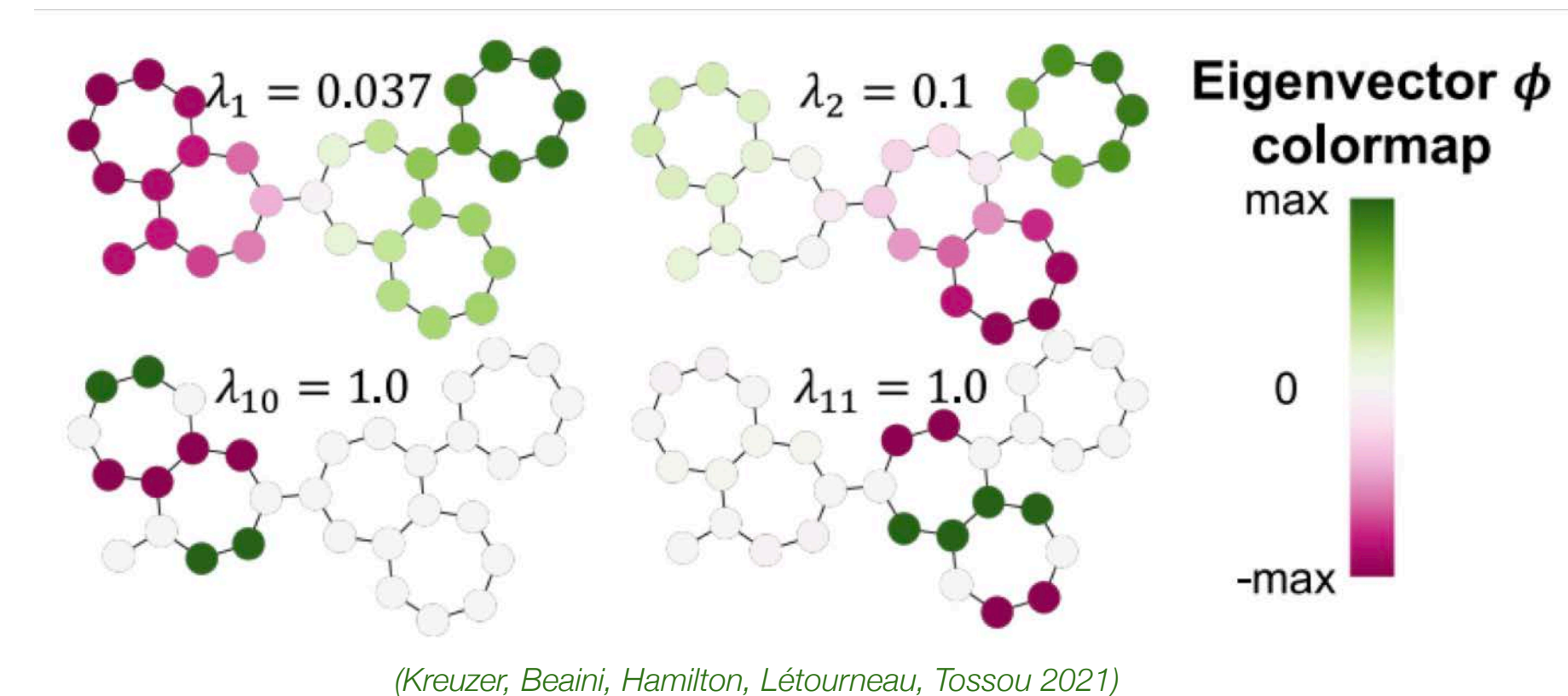
- Add node input features that encode “position” in the graph

- For instance: eigenvectors of the graph Laplacian (or its normalized versions)

$$\mathbf{L} = \mathbf{D} - \mathbf{A}$$

Diagonal matrix with node degrees *Adjacency matrix*

- adds global structural information
- Challenge: ambiguities (sign flips, eigenvalue multiplicities)

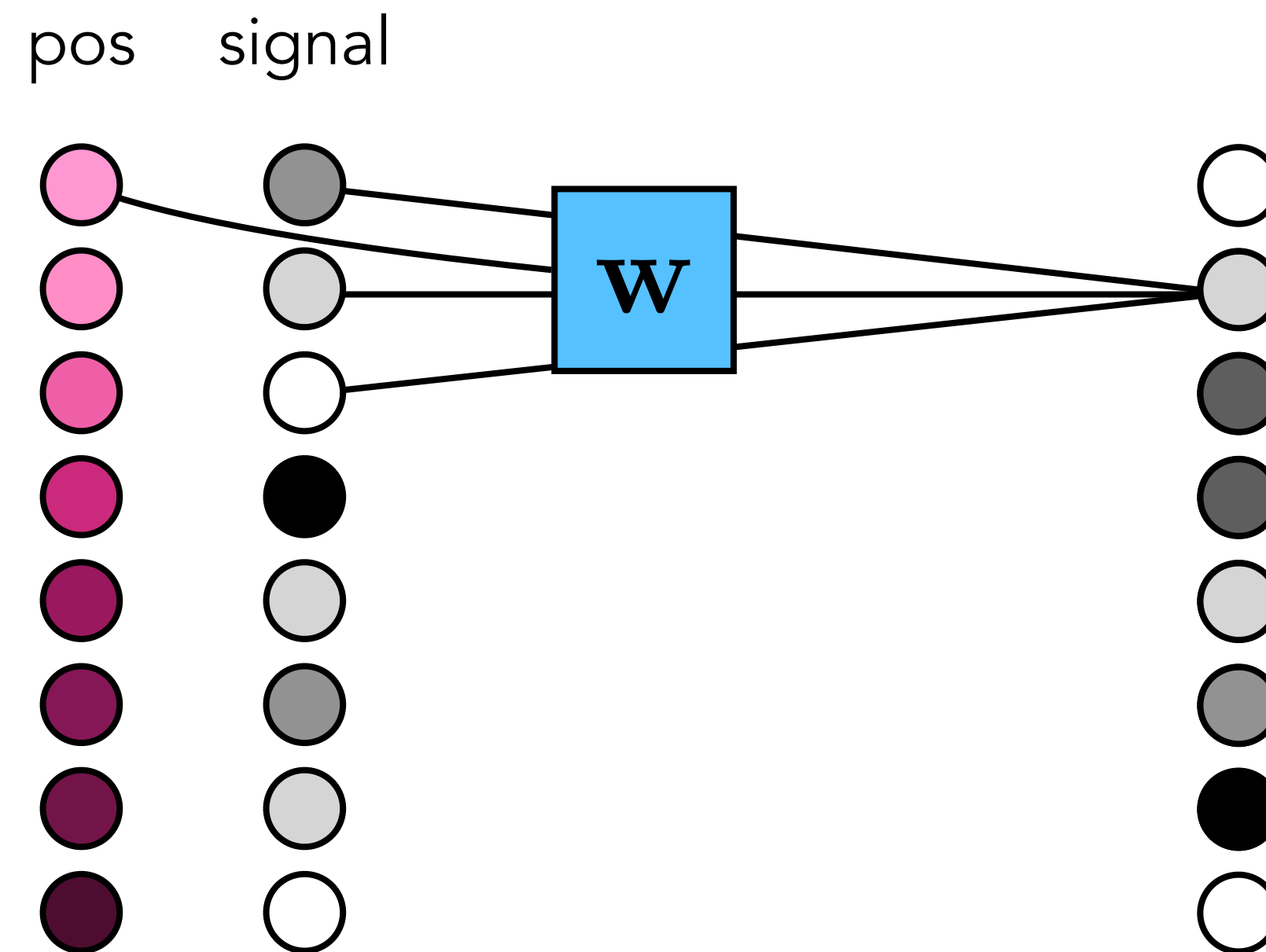


Graphs © Kreuzer, et al and Lim, et al. All rights reserved.
This content is excluded from our Creative Commons
license. For more information, see [https://ocw.mit.edu/
help/faq-fair-use/](https://ocw.mit.edu/help/faq-fair-use/)

Call back to CNNs:

What if you *don't* want to be shift invariant?

1. Use an architecture that is not shift invariant (e.g., MLP)
2. Add location information to the *input* to the convolutional filters — this is called **positional encoding**



Summary

- Encodes graph structure and node/edge attributes
- Important: permutation invariance/equivariance
- Main idea: **message passing** and **aggregations**
- **Can take graphs of varying size and structure** (similar to CNNs)
- Connections: graph signal processing, graphical models, distributed computing, isomorphism testing, ...
- Representational enhancements: higher-order, node IDs/augmentation

Appendix

- Graph Laplacian (unnormalized): degree matrix $\mathbf{D}$ - adjacency matrix $\mathbf{A}$

$$\mathbf{L} = \mathbf{D} - \mathbf{A}$$

$$\mathbf{D}_{ii} = \deg(v_i) \text{ or } \sum_{(v_i, v_j) \in E} w_{ij} \quad \text{and} \quad \mathbf{D}_{ij} = 0 \text{ for } i \neq j$$

- normalized: $\mathbf{I} - \mathbf{D}^{-1}\mathbf{A}$ or $\mathbf{I} - \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$

MIT OpenCourseWare

<https://ocw.mit.edu>

6.7960 Deep Learning

Fall 2024

For information about citing these materials or our Terms of Use, visit: <https://ocw.mit.edu/terms>